

MI ISP API

Version 3.4

© 2022 SigmaStar Technology Corp. All rights reserved.

SigmaStar Technology makes no representations or warranties including, for example but not limited to, warranties of merchantability, fitness for a particular purpose, non-infringement of any intellectual property right or the accuracy or completeness of this document, and reserves the right to make changes without further notice to any products herein to improve reliability, function or design. No responsibility is assumed by SigmaStar Technology arising out of the application or use of any product or circuit described herein; neither does it convey any license under its patent rights, nor the rights of others.

SigmaStar is a trademark of SigmaStar Technology Corp. Other trademarks or names herein are only for identification purposes only and owned by their respective owners.

REVISION HISTORY

Revision No.	Description	Date
3.0	Initial release	12/04/2020
3.1	- Added new API: MI_ISP_SetCustSegAttr MI_ISP_GetCustSegAttr MI_ISP_GetCustSegBuf MI_ISP_PutCustSegBuf - Update API: MI_ISP_SetChnParam	07/25/2021
	Added PROCFS INTRODUCTION	08/25/2021
3.2	- Added new API: MI_ISP_GetSubChnId	11/23/2021
3.3	Added Mochi Info	02/17/2022
3.4	Added Maruko info	03/21/2022

TABLE OF CONTENTS

REVISION HISTORY	i
TABLE OF CONTENTS	ii
1. 概述	1
1.1. 模块说明	1
1.2. 流程框图	1
1.2.1. Tiramisu 框图	1
1.2.2. Muffin 框图	1
1.2.3. Mochi 框图	2
1.2.4. Maruko 框图	2
1.3. 关键字说明	2
1.4. insmod 参数说明	3
2. API 参考	5
2.1. MI_ISP_CreateDevice	7
2.2. MI_ISP_DestroyDevice	9
2.3. MI_ISP_CreateChannel	10
2.4. MI_ISP_DestroyChannel	11
2.5. MI_ISP_SetInputPortCrop	11
2.6. MI_ISP_GetInputPortCrop	12
2.7. MI_ISP_SetChnParam	13
2.8. MI_ISP_GetChnParam	14
2.9. MI_ISP_StartChannel	15
2.10. MI_ISP_StopChannel	15
2.11. MI_ISP_SetOutputPortParam	16
2.12. MI_ISP_GetOutputPortParam	17
2.13. MI_ISP_EnableOutputPort	18
2.14. MI_ISP_DisableOutputPort	19
2.15. MI_ISP_Alloc_IQDataBuf	20
2.16. MI_ISP_Free_IQDataBuf	21
2.17. MI_ISP_CallBackTask_Register	22
2.18. MI_ISP_CallBackTask_Unregister	24
2.19. MI_ISP_SkipFrame	24
2.20. MI_ISP_LoadPortZoomTable	25
2.21. MI_ISP_StartPortZoom	26
2.22. MI_ISP_StopPortZoom	27
2.23. MI_ISP_GetPortCurZoomAttr	28
2.24. MI_ISP_SetCustSegAttr	29
2.25. MI_ISP_GetCustSegAttr	33
2.26. MI_ISP_GetCustSegBuf	34
2.27. MI_ISP_PutCustSegBuf	35
2.28. MI_ISP_GetSubChnId	36
3. ISP 数据类型	38
3.1. MI_ISP_DEV	39
3.2. MI_ISP_CHANNEL	39

3.3.	MI_ISP_PORT.....	39
3.4.	MI_ISP_DevMaskId_e.....	40
3.5.	MI_ISP_DevAttr_t.....	40
3.6.	MI_ISP_HDRTYPE_e.....	41
3.7.	MI_ISP_3DNR_Level_e.....	41
3.8.	MI_ISP_BindSnrId_e.....	42
3.9.	MI_ISP_SYNC3A_e.....	44
3.10.	MI_ISP_IQApiHeader_t.....	44
3.11.	MI_ISP_VersionPara_t.....	45
3.12.	MI_ISP_CustIQParam_t.....	45
3.13.	MI_ISP_ChannelAttr_t.....	46
3.14.	MI_ISP_ChnParam_t.....	47
3.15.	MI_ISP_OutPortParam_t.....	47
3.16.	MI_ISP_CALLBK_FUNC.....	48
3.17.	MI_ISP_CallBackMode_e.....	48
3.18.	MI_ISP_IrqType_e.....	49
3.19.	MI_ISP_CallBackParam_t.....	49
3.20.	MI_ISP_ZoomEntry_t.....	50
3.21.	MI_ISP_ZoomTable_t.....	51
3.22.	MI_ISP_ZoomAttr_t.....	51
3.23.	MI_ISP_CustSegAttr_t.....	52
3.24.	MI_ISP_CustSegInPortParam_t.....	52
3.25.	MI_ISP_CustSegOutPortParam_t.....	53
3.26.	MI_ISP_CustSegMode_e.....	54
3.27.	MI_ISP_InternalSeg_e.....	54
4.	ISP 错误码.....	56
5.	PROCFS 介绍.....	57
5.1.	cat.....	57
5.2.	echo.....	59

1. 概述

1.1. 模块说明

图像信号处理（Image Signal Processing），实现 HDR，3D/2D 降噪，3A 算法，WDR 等相关功能。

1.2. 流程框图

1.2.1. Tiramisu 框图

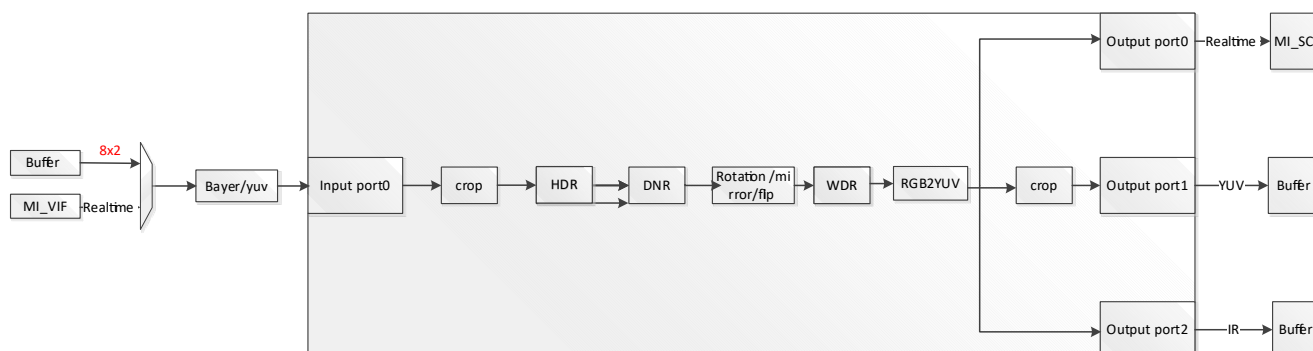


图 1-1: Tiramisu 框图

※ 注意

- Tiramisu 只有一个 Device0, 每个 Device 最多支持 16 个 channel。
- 只支持 YUV422_YUYV/UYVY 和 bayer 格式输入，只有 bayer 格式可以支持 3DNR 和 3A 功能。

1.2.2. Muffin 框图

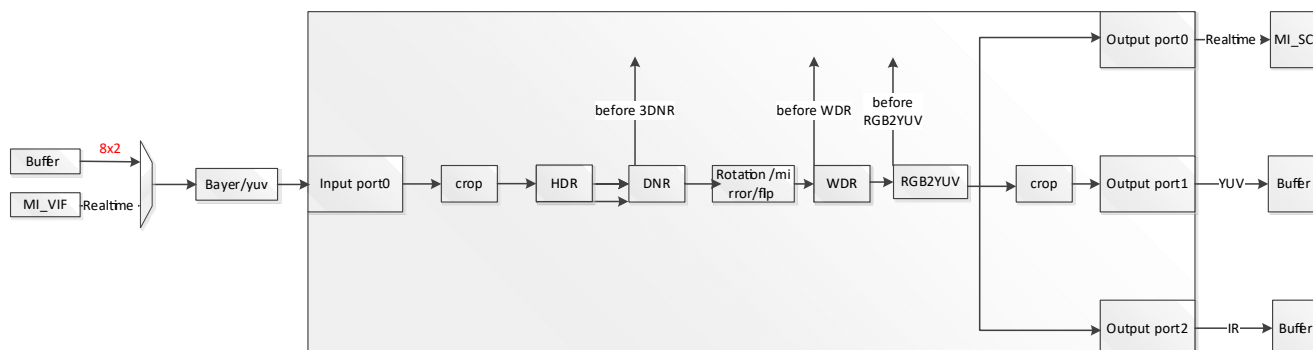


图 1-2: Muffin 框图

※ 注意

- Muffin 有两个 Device。Device0 和 Device1, 每个 Device 最多支持 16 个 channel。
- 只支持 YUV422_YUYV/UYVY 和 bayer 格式输入。
- 通过 MI_ISP_SetCustSegAttr 接口，使能 AI ISP 后，支持 app 从 before 3DNR/before WDR/before RGB2YUV 取出 buffer，并对 buffer 处理完成后，塞回给 before 3DNR/before WDR/before RGB2YUV。

- Before 3DNR，就是进入 DNR 模块之前的数据。
- AI ISP 功能，指的是 app 从 MI ISP 中取出 buffer，使用算法对 buffer 进行图像优化处理后，塞回给 MI ISP。

1.2.3. Mochi 框图

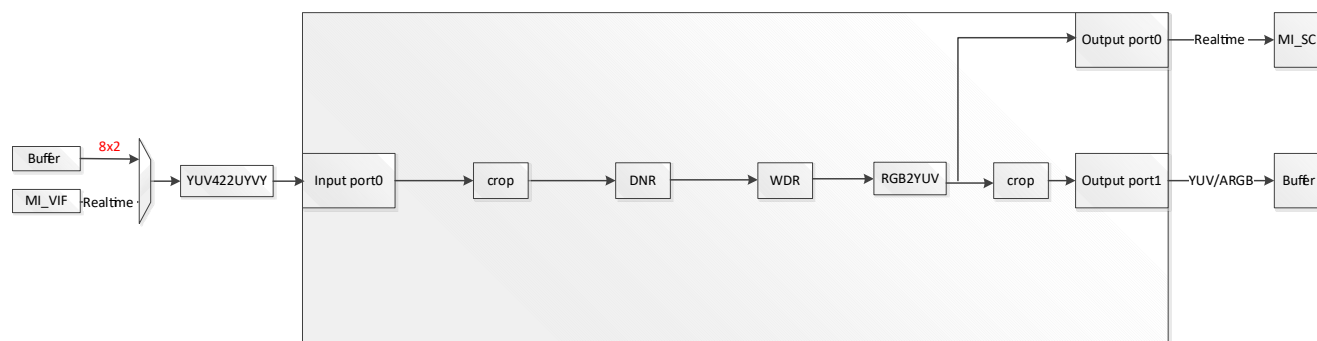


图 1-3: Mochi 框图

※ 注意

- Mochi 只有一个 Device0, 每个 Device 最多支持 16 个 channel。
- 只支持 YUV422_UYVY 格式输入。
- 不支持 HDR/Rotation/Mirror/Flip 功能。

1.2.4. Maruko 框图

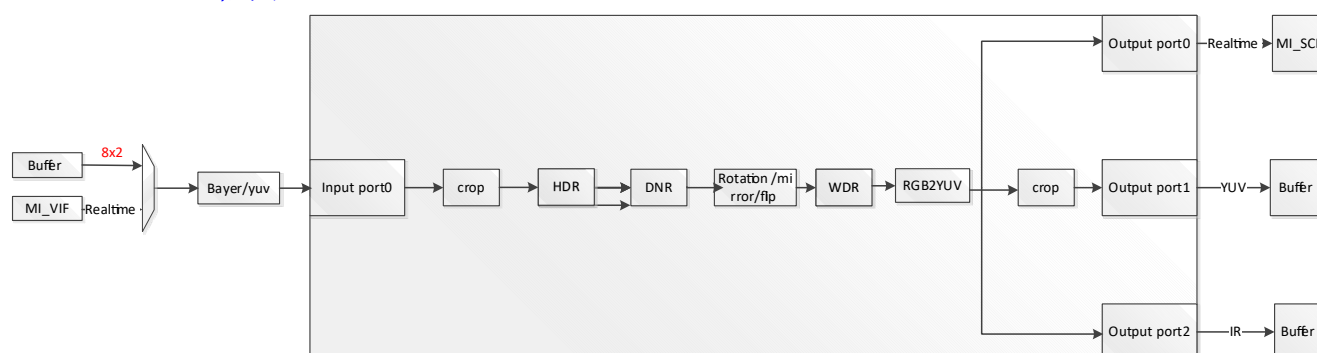


图 1-4: Maruko 框图

※ 注意

- Maruko 只有一个 Device0, 每个 Device 最多支持 16 个 channel。
- 只支持 YUV422_YUYV/UYVY 和 bayer 格式输入，只有 bayer 格式可以支持 3DNR/3A/Rot 功能。

1.3. 关键字说明

- Device
硬件设备。
- channel

Device 上的分时复用通道。

- DNR

Digital Noise Reduction，数字降噪技术。

2D 降噪：将一个像素与其周围像素平均，平均后噪声降低，缺点是会造成画面模糊；

3D 降噪：增添了时域处理，2D 降噪只考虑一帧图像，而 [3D 降噪](#) 进一步考虑帧与帧之间的时域关系，对每个像素进行时域上的平均。

- 3A 算法

AE (Auto Exposure，自动曝光)、AWB(Auto White Balance，自动白平衡)和 AF(Auto Focus，自动对焦)。

- HDR

High-Dynamic Range，高动态范围图像。

据不同的[曝光时间](#)的 LDR (Low-Dynamic Range，低动态范围图像)，并利用每个曝光时间相对应最佳细节的 LDR 图像来合成最终 HDR 图像。能够更好的反映出真实环境中的视觉效果。

- WDR

Wide Dynamic Range，宽动态范围。

开启后场景中特别亮和特别暗的部位都能看得清楚。宽动态范围是图像能分辨的最亮的亮度信号值与最暗的亮光信号值的比值。

- Rotation

旋转，将原始图像绕中心点旋转 0°/90°/180°/270°。

- mirror

水平镜像翻转。

- Flip

上下镜像翻转。

- Crop

裁剪，对输入图像进行裁剪。

- seg

分段模块，segment 的缩写。

1.4. insmod 参数说明

使用 insmod mi_isp.ko 后面可以带的参数使用情况如下所示：

表 1-1 insmod mi_isp.ko 可加参数

参数	含义	取值
g_u32DefaultDropNum	drop MI_ISP_CreateChannel 之后 buffer 的张数	1.default:10 由于 ISP 需要收敛，默认会 drop 掉 10 张 2.可以用户设定任意值

参数	含义	取值
		在快启的场景时，减少 drop 的张数，可以缩短启动时间

2. API 参考

该功能模块提供以下 API:

表 2-2: API 参考

API名	功能
MI_ISP_CreateDevice	创建一个 ISP 设备
MI_ISP_DestroyDevice	销毁一个 ISP 设备
MI_ISP_CreateChannel	创建一个 ISP 通道
MI_ISP_DestroyChannel	销毁一个 ISP 通道
MI_ISP_SetInputPortCrop	设定 ISP 输入端口裁剪参数
MI_ISP_GetInputPortCrop	获取 ISP 输入端口裁剪参数
MI_ISP_SetChnParam	设定 ISP 通道参数
MI_ISP_GetChnParam	获取 ISP 通道参数
MI_ISP_StartChannel	启动 ISP 通道
MI_ISP_StopChannel	停止 ISP 通道
MI_ISP_SetOutputPortParam	设定 ISP 输出端口参数
MI_ISP_GetOutputPortParam	获取 ISP 输出端口参数
MI_ISP_EnableOutputPort	启用 ISP 输出端口
MI_ISP_DisableOutputPort	禁用 ISP 输出端口
MI_ISP_Alloc_IQDataBuf	申请 IQ 数据缓存区
MI_ISP_Free_IQDataBuf	释放 IQ 数据缓存区
MI_ISP_CallbackTask_Register	注册 ISP 回调接口
MI_ISP_CallbackTask_Unregister	注销 ISP 回调接口
MI_ISP_SkipFrame	设定跳过 FrameNum
MI_ISP_LoadPortZoomTable	载入 ISP 端口 Zoom Table
MI_ISP_StartPortZoom	启动 ISP 端口 Zoom
MI_ISP_StopPortZoom	停止 ISP 端口 Zoom
MI_ISP_GetPortCurZoomAttr	获取 ISP 端口实时的 Zoom 属性
MI_ISP_SetCustSegAttr	设置AI ISP属性
MI_ISP_GetCustSegAttr	获取AI ISP属性
MI_ISP_GetCustSegBuf	获取AI ISP buffer
MI_ISP_PutCustSegBuf	释放AI ISP buffer

API名	功能
MI_ISP_GetSubChnId	获取ISP的子通道号

2.1. MI_ISP_CreateDevice

➤ 功能

创建一个 ISP 设备。

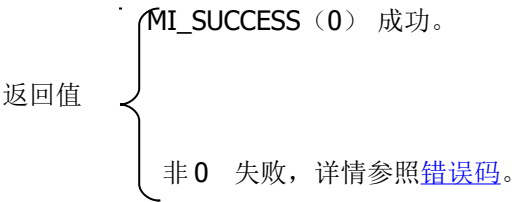
➤ 语法

```
MI_S32 MI_ISP_CreateDevice(MI_ISP_DEV DevId, MI_ISP_DevAttr_t *pstDevAttr);
```

➤ 形参

参数名称	描述	输入/输出
DevId	ISP 设备号。	输入
pstDevAttr	ISP 设备属性指针。静态属性。	输入

➤ 返回值



➤ 依赖

- 头文件：mi_isp_datatype.h、mi_isp.h
- 库文件：libmi_isp.a

※ 注意

- 在调用前要保证 ISP 设备处于禁用状态。如果 ISP 设备已处于使能状态，可以使用 [MI_ISP_DestroyDevice](#) 来禁用设备。

➤ 举例

```
MI_ISP 初始化：
MI_S32 ST_IspModuleInit(MI_ISP_DEV IspDevId)
{
#define ST_MAX_ISP_OUTPORT_NUM (2)
    MI_ISP_CHANNEL IspChnId = 0;
    MI_ISP_PORT IspOutPortId =0;

    MI_ISP_DevAttr_t stCreateDevAttr;
    memset(&stCreateDevAttr, 0x0, sizeof(MI_ISP_DevAttr_t));
    stCreateDevAttr.u32DevStitchMask = E_MI_ISP_DEVICE_ID0;

    MI_ISP_CreateDevice(IspDevId, &stCreateDevAttr);

    MI_ISP_ChannelAttr_t stIspChnAttr;
    memset(&stIspChnAttr, 0x0, sizeof(MI_ISP_ChannelAttr_t));
```

```

stIspChnAttr.u32SensorBindId = E_MI_ISP_SENSOR0;
MI\_ISP\_CreateChannel(IspDevId, IspChnId, &stIspChnAttr);

MI_SYS_WindowRect_t stInputCropInfo;
stInputCropInfo.u16x=0;
stInputCropInfo.u16y=0;
stInputCropInfo.u16width=0;
stInputCropInfo.u16height=0;//width/height==0 no use crop.
MI\_ISP\_SetInputPortCrop(IspDevId, IspChnId, &stInputCropInfo);

MI\_ISP\_ChnParam\_t stChnParam;
stChnParam.eHDRType = E_MI_ISP_HDR_TYPE_OFF;
stChnParam.e3DNRLevel = E_MI_ISP_3DNR_LEVEL2;
stChnParam.bMirror = FALSE;
stChnParam.bFlip = FALSE;
stChnParam.eRot = E_MI_SYS_ROTATE_NONE;
//use rot or flip, 3dnr level must more than 0
stChnParam.bY2bEnable = FALSE;
MI\_ISP\_SetChnParam(IspDevId, IspChnId, &stChnParam);
MI\_ISP\_StartChannel(IspDevId, IspChnId);

for(IspOutPortId=0; IspOutPortId<ST_MAX_ISP_OUTPORT_NUM; IspOutPortId++)
{
    MI\_ISP\_OutPortParam\_t stIspOutputParam;
    memset(&stIspOutputParam, 0x0, sizeof(MI_ISP_OutPortParam_t));
    stIspOutputParam.stCropRect.u16x=0;
    stIspOutputParam.stCropRect.u16y=0;
    stIspOutputParam.stCropRect.u16Width=0;
    stIspOutputParam.stCropRect.u16Height=0;//width/height use 0, not use crop
    stIspOutputParam.ePixelFormat =
E_MI_SYS_PIXEL_FRAME_YUV_SEMIPLANAR_YUV420;
    MI\_ISP\_SetOutputPortParam(IspDevId, IspChnId, IspOutPortId,
&stIspOutputParam);
    MI\_ISP\_EnableOutputPort(IspDevId, IspChnId, IspOutPortId);
}

return MI_SUCCESS;
}

```

MI_ISP 去初始化:

```

MI_S32 ST_IspModuleUnInit(MI_ISP_DEV IspDevId)
{
#define ST_MAX_ISP_OUTPORT_NUM (2)

```

```

MI_ISP_CHANNEL IspChnId = 0;
MI_ISP_PORT IspOutPortId = 0;

for(IspOutPortId=0; IspOutPortId<ST_MAX_ISP_OUTPORT_NUM; IspOutPortId++)
{
    MI\_ISP\_DisableOutputPort(IspDevId, IspChnId, IspOutPortId);
}

MI\_ISP\_StopChannel(IspDevId, IspChnId);
MI\_ISP\_DestroyChannel(IspDevId, IspChnId);

MI\_ISP\_DestroyDevice(IspDevId);

return MI_SUCCESS;
}

```

➤ 相关主题

[MI_ISP_DestroyDevice](#)

2.2. MI_ISP_DestroyDevice

➤ 功能

销毁一个 ISP 设备。

➤ 语法

```
MI_S32 MI_ISP_DestroyDevice(MI_ISP_DEV DevId);
```

➤ 形参

参数名称	描述	输入/输出
DevId	ISP 设备号。	输入

➤ 返回值

返回值 {
 MI_SUCCESS (0) 成功。
 非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件: mi_isp_datatype.h、mi_isp.h
- 库文件: libmi_isp.a

※ 注意

需要先禁用所有的输出端口，然后停下 **channel**，销毁通道，才能销毁设备。

➤ 举例

参考 [MI_ISP_CreateDevice](#) 举例。

➤ 相关主题

[MI_ISP_CreateDevice](#)

2.3. MI_ISP_CreateChannel

➤ 功能

创建一个 ISP 通道。

➤ 语法

```
MI_S32 MI_ISP_CreateChannel(MI_ISP_DEV DevId, MI_ISP_CHANNEL ChnId,
MI\_ISP\_ChannelAttr\_t *pstChAttr);
```

➤ 形参

参数名称	描述	输入/输出
DevId	ISP 设备号。	输入
ChnId	ISP 通道号	输入
pstChAttr	ISP 通道属性指针	输入

➤ 返回值

返回值 { MI_SUCCESS (0) 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_isp_datatype.h、mi_isp.h
- 库文件：libmi_isp.a

※ 注意

需要先创建设备，才能创建通道。

➤ 举例

参考 [MI_ISP_CreateDevice](#) 举例。

➤ 相关主题

[MI_ISP_DestroyChannel](#)。

2.4. MI_ISP_DestroyChannel

➤ 功能

销毁一个 ISP 通道。

➤ 语法

```
MI_S32 MI_ISP_DestroyChannel(MI_ISP_DEV DevId, MI_ISP_CHANNEL ChnId);
```

➤ 形参

参数名称	描述	输入/输出
DevId	ISP 设备号。	输入
ChnId	ISP 通道号	输入

➤ 返回值

返回值 {
MI_SUCCESS (0) 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_isp_datatype.h、mi_isp.h
- 库文件：libmi_isp.a

※ 注意

需要先禁用所有的输出端口，然后停下 channel，才能销毁通道。

➤ 举例

参考 [MI_ISP_CreateDevice](#) 举例。

➤ 相关主题

[MI_ISP_CreateChannel](#)

2.5. MI_ISP_SetInputPortCrop

➤ 功能

设定 ISP 输入端口裁剪参数。

➤ 语法

```
MI_S32 MI_ISP_SetInputPortCrop(MI_ISP_DEV DevId, MI_ISP_CHANNEL ChnId,  
MI_SYS_WindowRect_t *pstCropInfo);
```

➤ 形参

参数名称	描述	输入/输出
DevId	ISP 设备号。	输入
ChnId	ISP 通道号	输入

pstCropInfo	ISP 输入端口裁剪参数指针	输入
-------------	----------------	----

➤ 返回值

返回值 { MI_SUCCESS (0) 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件: mi_isp_datatype.h、mi_isp.h
- 库文件: libmi_isp.a

※ 注意

- 只有 input 是 Dram buffer 时，才支持 input crop。
- Crop x、Crop y、width、height 都是 2 对齐
- Isp 内部的 3A 只会统计裁剪区域的信息，会影响到整个画面最终 3A 计算结果。
- 当输入数据压缩模式不为 E_MI_SYS_COMPRESS_MODE_NONE 时，输入数据不支持裁剪。

➤ 举例

无。

➤ 相关主题

无。

2.6. MI_ISP_GetInputPortCrop

➤ 功能

获取 ISP 输入端口裁剪参数。

➤ 语法

```
MI_S32 MI_ISP_GetInputPortCrop(MI_ISP_DEV DevId, MI_ISP_CHANNEL ChnId,
MI_SYS_WindowRect_t *pstCropInfo);
```

➤ 形参

参数名称	描述	输入/输出
DevId	ISP 设备号。	输入
ChnId	ISP 通道号	输入
pstCropInfo	ISP 输入端口裁剪参数指针	输出

➤ 返回值

返回值 { MI_SUCCESS (0) 成功。
非 0 失败，详情参照[错误码](#)。

- 依赖
 - 头文件：mi_isp_datatype.h、mi_isp.h
 - 库文件：libmi_isp.a

※ 注意
无。

➤ 举例
无。

➤ 相关主题
无。

2.7. MI_ISP_SetChnParam

➤ 功能
设定 ISP 通道参数。

➤ 语法
MI_S32 MI_ISP_SetChnParam(MI_ISP_DEV DevId, MI_ISP_CHANNEL ChnId, [MI_ISP_ChnParam_t](#) *pstChnParam);

➤ 形参

参数名称	描述	输入/输出
DevId	ISP 设备号。	输入
ChnId	ISP 通道号	输入
pstChnParam	ISP 通道参数指针	输入

➤ 返回值

返回值

MI_SUCCESS (0) 成功。

非 0 失败，详情参照[错误码](#)。

- 依赖
 - 头文件：mi_isp_datatype.h、mi_isp.h
 - 库文件：libmi_isp.a

※ 注意
建议在通道停止的情况下切换通道参数，避免通道内缓存的 buffer 和后端参数不匹配，导致异常。

➤ 举例

参考 [MI_ISP_CreateDevice](#) 举例。

- 相关主题
[MI_ISP_GetChnParam](#)

2.8. MI_ISP_GetChnParam

- 功能
获取 ISP 通道参数。
- 语法
MI_S32 MI_ISP_GetChnParam(MI_ISP_DEV DevId, MI_ISP_CHANNEL ChnId, [MI_ISP_ChnParam_t](#) *pstChnParam);

- 形参

参数名称	描述	输入/输出
DevId	ISP 设备号。	输入
ChnId	ISP 通道号	输入
pstChnParam	ISP 通道参数指针	输出

- 返回值

返回值 { MI_SUCCESS (0) 成功。
非 0 失败，详情参照[错误码](#)。

- 依赖
 - 头文件：mi_isp_datatype.h、mi_isp.h
 - 库文件：libmi_isp.a

- ※ 注意
无。

- 举例
无。

- 相关主题
[MI_ISP_SetChnParam](#)

2.9. MI_ISP_StartChannel

➤ 功能

启动 ISP 通道。

➤ 语法

```
MI_S32 MI_ISP_StartChannel(MI_ISP_DEV DevId, MI_ISP_CHANNEL ChnId);
```

➤ 形参

参数名称	描述	输入/输出
DevId	ISP 设备号。	输入
ChnId	ISP 通道号	输入

➤ 返回值

返回值 {
MI_SUCCESS (0) 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_isp_datatype.h、mi_isp.h
- 库文件：libmi_isp.a

※ 注意

无。

➤ 举例

参考 [MI_ISP_CreateDevice](#) 举例。

➤ 相关主题

[MI_ISP_SetChnParam](#)

2.10. MI_ISP_StopChannel

➤ 功能

停止 ISP 通道。

➤ 语法

```
MI_S32 MI_ISP_StopChannel(MI_ISP_DEV DevId, MI_ISP_CHANNEL ChnId);
```

➤ 形参

参数名称	描述	输入/输出
DevId	ISP 设备号。	输入
ChnId	ISP 通道号	输入

➤ 返回值

返回值	MI_SUCCESS (0) 成功。
	非 0 失败，详情参照 错误码 。

➤ 依赖

- 头文件：mi_isp_datatype.h、mi_isp.h
- 库文件：libmi_isp.a

※ 注意

在通道创建的情况下，都可以停下通道，通道停下所有的输出端口都不会有数据输出。

➤ 举例

参考 MI_ISP_CreateDevice 举例。

➤ 相关主题

无。

2.11. MI_ISP_SetOutputPortParam

➤ 描述

设定 ISP 输出端口参数。

➤ 语法

MI_S32 MI_ISP_SetOutputPortParam(MI_ISP_DEV DevId, MI_ISP_CHANNEL ChnId, MI_ISP_PORT PortId, [MI_ISP_OutPortParam_t](#) *pstOutPortParam);

➤ 形参

参数名称	描述	输入/输出
DevId	ISP 设备号。	输入
ChnId	ISP 通道号	输入
PortId	ISP 端口号	输入
pstOutPortParam	ISP 输出端口参数指针	输入

➤ 返回值

	MI_SUCCESS (0) 成功。

返回值

非 0 失败，详情参照[错误码](#)。

- 依赖
 - 头文件：mi_isp_datatype.h、mi_isp.h
 - 库文件：libmi_isp.a
- ※ 注意
 - Output port0 只支持和 MI_SCL realtime 绑定，输出区域和 input 相同，可以不调用此 api 设置。
 - Output port1 支持 crop，需要指定 size 输出。
 - Output port2 只支持输出 RGBIR sensor 的 IR 数据，Output W/H 固定为 input W/H 的各一半，可以不调用此 api 设置。
- 举例

参考 [MI_ISP_CreateDevice](#) 举例。
- 相关主题

无。

2.12. MI_ISP_GetOutputPortParam

- 描述

获取 ISP 输出端口参数。
- 语法

```
MI_S32 MI_ISP_GetOutputPortParam(MI_ISP_DEV DevId, MI_ISP_CHANNEL ChnId, MI_ISP_PORT PortId, MI\_ISP\_OutPortParam\_t *pstOutPortParam);
```
- 形参

参数名称	描述	输入/输出
DevId	ISP 设备号。	输入
ChnId	ISP 通道号	输入
PortId	ISP 端口号	输入
pstOutPortParam	ISP 输出端口参数指针	输出
- 返回值

返回值 {
 - MI_SUCCESS (0) 成功。
 - 非 0 失败，详情参照[错误码](#)。}
- 依赖
 - 头文件：mi_isp_datatype.h、mi_isp.h

- 库文件：libmi_isp.a

※ 注意
无。

➤ 举例
无。

➤ 相关主题
[MI_ISP_SetOutputPortParam](#)

2.13. MI_ISP_EnableOutputPort

➤ 描述
启用 ISP 输出端口。

➤ 语法
MI_S32 MI_ISP_EnableOutputPort(MI_ISP_DEV DevId, MI_ISP_CHANNEL ChnId,
MI_ISP_PORT PortId);

➤ 形参

参数名称	描述	输入/输出
DevId	ISP 设备号。	输入
ChnId	ISP 通道号	输入
PortId	ISP 端口号	输入

➤ 返回值

返回值 { MI_SUCCESS (0) 成功。
非 0 失败，详情参照[错误码](#)。

- 依赖
- 头文件：mi_isp_datatype.h、mi_isp.h
 - 库文件：libmi_isp.a

※ 注意

- Output port1 必须要在设置了 output port 属性的前提下才可以使能该端口。
- Output port2 只有在使用 RGBIR sensor 的时候才可以使能。

➤ 举例
参考 [MI_ISP_CreateDevice](#) 举例。

➤ 相关主题

[MI_ISP_DisableOutputPort](#)

2.14. MI_ISP_DisableOutputPort

➤ 描述

禁用 ISP 输出端口。

➤ 语法

```
MI_S32 MI_ISP_DisableOutputPort(MI_ISP_DEV DevId, MI_ISP_CHANNEL ChnId,  
MI_ISP_PORT PortId);
```

➤ 形参

参数名称	描述	输入/输出
DevId	ISP 设备号。	输入
ChnId	ISP 通道号	输入
PortId	ISP 端口号	输入

➤ 返回值

返回值 { MI_SUCCESS (0) 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_isp_datatype.h、mi_isp.h
- 库文件：libmi_isp.a

※ 注意

无。

➤ 举例

无。

➤ 相关主题

无。

2.15. MI_ISP_Alloc_IQDataBuf

➤ 描述

申请 IQ API 数据缓存区。

➤ 语法

MI_S32 MI_ISP_Alloc_IQDataBuf(MI_U32 u32Size,void **pUserVirAddr);

➤ 形参

参数名称	描述	输入/输出
u32Size	IQ 数据缓存区大小。	输入
pUserVirAddr	IQ 数据缓存区指针地址	输入

➤ 返回值

返回值 { MI_SUCCESS (0) 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_isp_datatype.h、mi_isp.h
- 库文件：libmi_isp.a

※ 注意

- 建议使用该 api 申请 IQ api 数据，避免高频率调用造成 cpu loading 较高风险。
- 一个线程申请一块 isp data buffer，避免线程间 buffer 共用，导致线程之间 buffer 互踩。

➤ 举例

```
#define MI_ISP_MAX_DATA_SIZE (80*1024)
MI_ISP_Alloc_IQDataBuf(MI_ISP_MAX_DATA_SIZE, &pIspBuffer);
MI_ISP_IQ_ColorToGrayType_t *pstColorToGray = (MI_ISP_IQ_ColorToGrayType_t *)pIspBuffer;
MI_ISP_IQ_GetColorToGray( Channel, pstColorToGray);
if(pstColorToGray->bEnable == SS_TRUE)
    pstColorToGray->bEnable = SS_FALSE;
else
    pstColorToGray->bEnable = SS_TRUE;
MI_ISP_IQ_SetColorToGray( Channel, pstColorToGray);

MI_ISP_IQ_ContrastType_t *pstContrast = (MI_ISP_IQ_ContrastType_t *)pIspBuffer;
MI_ISP_IQ_GetContrast( Channel, pstContrast);
MI_ISP_IQ_SetContrast( Channel, pstContrast);
```

MI_ISP_Free_IQDataBuf(pIspBuffer)

- 相关主题
[MI_ISP_Free_IQDataBuf](#)。

2.16. MI_ISP_Free_IQDataBuf

- 描述
释放 IQ 数据缓存区。

- 语法
MI_S32 MI_ISP_Free_IQDataBuf(void *pUserBuf);

- 形参

参数名称	描述	输入/输出
pUserBuf	IQ 申请的数据缓存区指针	输入

- 返回值

返回值 { MI_SUCCESS (0) 成功。
非 0 失败，详情参照[错误码](#)。

- 依赖
 - 头文件：mi_isp_datatype.h、mi_isp.h
 - 库文件：libmi_isp.a

- ※ 注意
无。

- 举例
参考 [MI_ISP_Alloc_IQDataBuf](#) 举例。

- 相关主题
无。

2.17. MI_ISP_CallbackTask_Register

➤ 描述

注册 ISP 回调接口。

➤ 语法

MI_S32 MI_ISP_CallbackTask_Register(MI_ISP_DEV DevId, [MI_ISP_CallbackParam_t](#) *pstCallbackParam);

➤ 形参

参数名称	描述	输入/输出
DevId	ISP 设备号。	输入
pstCallbackParam	ISP 回调参数	输入

➤ 返回值

返回值 {
MI_SUCCESS (0) 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_isp_datatype.h, mi_isp.h
- 库文件：libmi_isp.a

※ 注意

- 该接口目前只支持 kernel mode 调用
- 和 [MI_ISP_CallbackTask_Unregister](#) 成对调用

➤ 举例

```
MI_S32 _MI_ISP_vsync1(MI_U64 u64Data)
{
    DBG_ERR("DATA %llu \n", u64Data);
    return 0;
}

MI_S32 _MI_ISP_vsync2(MI_U64 u64Data)
{
    DBG_ERR("DATA %llu \n", u64Data);
    return 0;
}

static MS_S32 _MI_ISP_testRegCallback(void)
{
```

```

    MI\_ISP\_CallbackParam\_t stCallbackParam1;
    MI\_ISP\_CallbackParam\_t stCallbackParam2;
    memset(&stCallbackParam1, 0x0, sizeof(MI\_ISP\_CallbackParam\_t));
    memset(&stCallbackParam2, 0x0, sizeof(MI\_ISP\_CallbackParam\_t));
    stCallbackParam1.eCallbackMode = E_MI_ISP_CALLBACK_ISR;
    stCallbackParam1.eIrqType = E_MI_ISP_IRQ_ISPVSYNC;
    stCallbackParam1.pfnCallbackFunc = _mi_isp_vsync1;
    stCallbackParam1.u64Data = 12;
    MI\_ISP\_CallbackTask\_Register(&stCallbackParam1);
    stCallbackParam2.eCallbackMode = E_MI_ISP_CALLBACK_ISR;
    stCallbackParam2.eIrqType = E_MI_ISP_IRQ_ISPVSYNC;
    stCallbackParam2.pfnCallbackFunc = _mi_isp_vsync2;
    stCallbackParam2.u64Data = 23;
    MI\_ISP\_CallbackTask\_Register(&stCallbackParam2);
    return 0;
}

static MS_S32 _MI_ISP_testUnRegCallback(void)
{
    MI\_ISP\_CallbackParam\_t stCallbackParam1;
    MI\_ISP\_CallbackParam\_t stCallbackParam2;
    memset(&stCallbackParam1, 0x0, sizeof(MI\_ISP\_CallbackParam\_t));
    memset(&stCallbackParam2, 0x0, sizeof(MI\_ISP\_CallbackParam\_t));
    stCallbackParam1.eCallbackMode = E_MI_ISP_CALLBACK_ISR;
    stCallbackParam1.eIrqType = E_MI_ISP_IRQ_ISPVSYNC;
    stCallbackParam1.pfnCallbackFunc = _mi_isp_vsync1;
    stCallbackParam1.u64Data = 12;
    MI\_ISP\_CallbackTask\_Unregister(&stCallbackParam1);
    stCallbackParam2.eCallbackMode = E_MI_ISP_CALLBACK_ISR;
    stCallbackParam2.eIrqType = E_MI_ISP_IRQ_ISPVSYNC;
    stCallbackParam2.pfnCallbackFunc = _mi_isp_vsync2;
    stCallbackParam2.u64Data = 23;
    MI\_ISP\_CallbackTask\_Unregister(&stCallbackParam2);
    return 0;
}

```

➤ 相关主题

[MI_ISP_CallbackTask_Unregister](#)

2.18. MI_ISP_CallbackTask_Unregister

➤ 描述

注销回调接口。

➤ 语法

```
MI_S32 MI_ISP_CallbackTask_Unregister(MI_ISP_DEV DevId, MI_ISP_CallbackParam_t *pstCallBackParam);
```

➤ 形参

参数名称	描述	输入/输出
DevId	ISP 设备号。	输入
pstCallBackParam	ISP 回调参数	输入

➤ 返回值

返回值 { MI_SUCCESS (0) 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_isp_datatype.h、mi_isp.h
- 库文件：libmi_isp.a

※ 注意

- 该接口目前只支持 kernel mode 调用
- 和 [MI_ISP_CallbackTask_Register](#) 成对调用

➤ 举例

见 [MI_ISP_CallbackTask_Register](#) 举例。

➤ 相关主题

无。

2.19. MI_ISP_SkipFrame

➤ 描述

设定从调用该 API 开始跳过 Frame count。

➤ 语法

```
MI_S32 MI_ISP_SkipFrame(MI_ISP_DEV DevId, MI_ISP_CHANNEL ChnId, MI_U32 u32FrameNum);
```

➤ 形参

参数名称	描述	输入/输出
ChnId	ISP 通道号	输入
u32FrameNum	Frame 数量	输入

➤ 返回值

返回值	MI_SUCCESS (0) 成功。
	非 0 失败，详情参照 错误码 。

➤ 依赖

- 头文件：mi_isp_datatype.h、mi_isp.h
- 库文件：libmi_isp.a

※ 注意

无。

➤ 举例

无。

➤ 相关主题

无。

2.20. MI_ISP_LoadPortZoomTable

➤ 描述

载入 ISP 端口 Zoom Table。

➤ 语法

MI_S32 MI_ISP_LoadPortZoomTable(MI_ISP_DEV DevId, MI_ISP_CHANNEL ChnId,
[MI_ISP_ZoomTable_t](#) *pZoomTable);

➤ 形参

参数名称	描述	输入/输出
DevId	ISP 设备号。	输入
ChnId	ISP 通道号	输入
pZoomTable	Zoom Table参数	输入

➤ 返回值

返回值 { MI_SUCCESS (0) 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi_isp_datatype.h、mi_isp.h
- 库文件：libmi_isp.a

※ 注意

- 通过该接口提前加载好 zoom table，可以每一帧 frame 依次应用 table 中的 zoom 参数，保证 zoom 效果平滑，对比应用循环 call MI_ISP_SetOutputPortParam 设置 crop，没有办法保证每一帧 frame 生效设置，无法做到平滑效果。

X0	Y0	W0	H0	SensorId=0
X1	Y1	W1	H1	SensorId=0
X2	Y2	W2	H2	SensorId=1
X3	Y3	W3	H3	SensorId=1

表中包含 crop 位置 and 对应 sensorid 信息，sensorid 信息以支持长短焦切换场景。

- Zoom 功能由 MI_ISP 和 MI_SCL 配合完成，MI_ISP 负责 zoom 过程中 3A 效果和长短焦切换场景时通知 sensor driver 切换 sensor。
- MI_SCL 负责 crop 和 scaling up，所以在 zoom 场景中，MI_SCL 不能再使用 crop 功能。
- Zoom 运行过程中，不可以重复 load table，需要先 [MI_ISP_StopPortZoom](#)，再 load table。

➤ 举例

无。

➤ 相关主题

无。

2.21. MI_ISP_StartPortZoom

➤ 描述

启动 ISP 端口 Zoom。

➤ 语法

MI_S32 MI_ISP_StartPortZoom(MI_ISP_DEV DevId, MI_ISP_CHANNEL ChnId, [MI_ISP_ZoomAttr_t](#) *pstZoomAttr);

➤ 形参

参数名称	描述	输入/输出
DevId	ISP 设备号。	输入

参数名称	描述	输入/输出
ChnId	ISP 通道号	输入
pstZoomAttr	Zoom Table参数	输入

➤ 返回值

返回值	MI_SUCCESS (0) 成功。
	非 0 失败，详情参照 错误码 。

➤ 依赖

- 头文件：mi_isp_datatype.h、mi_isp.h
- 库文件：libmi_isp.a

※ 注意

X0	Y0	W0	H0	SensorId=0
X1	Y1	W1	H1	SensorId=0
X2	Y2	W2	H2	SensorId=1
X3	Y3	W3	H3	SensorId=1

u32FromEntryIndex 和 u32ToEntryIndex 为 table 中 index0~3 任意一个。
当 u32FromEntryIndex != u32ToEntryIndex，逐帧应用从 u32FromEntryIndex 到 u32ToEntryIndex 的 crop 参数。
当 u32FromEntryIndex == u32ToEntryIndex，会停在当前 u32FromEntryIndex 参数。
当 MI 内部检测到下一个 entry 的 sensorid 和当前 sensor id 不一致时，当前帧做完后会将 sensirid 通知给 sensor driver，切换到对应 sensor 上。

➤ 举例

无。

➤ 相关主题

[MI_ISP_StopPortZoom](#)

2.22. MI_ISP_StopPortZoom

➤ 描述

停止 ISP 端口 Zoom。

➤ 语法

MI_S32 MI_ISP_StopPortZoom(MI_ISP_DEV DevId, MI_ISP_CHANNEL ChnId);

➤ 形参

参数名称	描述	输入/输出
DevId	ISP 设备号。	输入
ChnId	ISP 通道号	输入

➤ 返回值

返回值	MI_SUCCESS (0) 成功。
	非 0 失败，详情参照 错误码 。

➤ 依赖

- 头文件：mi_isp_datatype.h、mi_isp.h
- 库文件：libmi_isp.a

※ 注意

和 [MI_ISP_StartPortZoom](#) 成对调用。

➤ 举例

无。

➤ 相关主题

无。

2.23. MI_ISP_GetPortCurZoomAttr

➤ 描述

获取 ISP 端口实时的 Zoom 属性。

➤ 语法

```
MI_S32 MI_ISP_GetPortCurZoomAttr(MI_ISP_DEV DevId, MI_ISP_CHANNEL ChnId,  
MI\_ISP\_ZoomAttr\_t *pstZoomAttr);
```

➤ 形参

参数名称	描述	输入/输出
DevId	ISP 设备号。	输入
ChnId	ISP 通道号	输入
pstZoomAttr	Zoom 属性状态	输出

- 返回值
- 返回值

MI_SUCCESS (0) 成功。

非 0 失败，详情参照[错误码](#)。
- 依赖
- 头文件：mi_isp_datatype.h、mi_isp.h
 - 库文件：libmi_isp.a
- ※ 注意
- 无。
- 举例
- 无。
- 相关主题
- 无。

2.24. MI_ISP_SetCustSegAttr

- 描述
- 设置 AI ISP 属性。
- 语法
- ```
MI_S32 MI_ISP_SetCustSegAttr(MI_ISP_DEV DevId, MI_ISP_CHANNEL ChnId,
MI_ISP_CustSegAttr_t *pstCustSegAttr);
```

- 形参

| 参数名称           | 描述       | 输入/输出 |
|----------------|----------|-------|
| DevId          | ISP 设备号。 | 输入    |
| ChnId          | ISP 通道号  | 输入    |
| pstCustSegAttr | AI ISP属性 | 输入    |

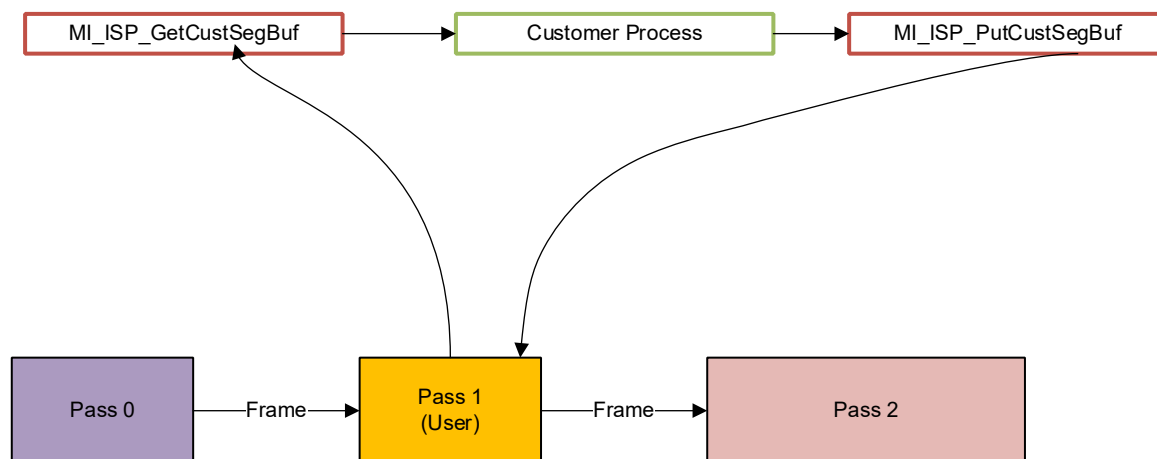
- 返回值
- 返回值

MI\_SUCCESS (0) 成功。

非 0 失败，详情参照[错误码](#)。
- 依赖
- 头文件：mi\_isp\_datatype.h、mi\_isp.h

- 库文件: libmi\_isp.a

※ 注意



pstCustSegAttr->eMode 的值为 [E\\_MI\\_ISP\\_CUST\\_SEG\\_MODE\\_INSERT](#) 或 [E\\_MI\\_ISP\\_CUST\\_SEG\\_MODE\\_REPLACE](#) 时, 会使能 AI ISP。

此时 MI ISP 内部会分成 3 个 pass。pass0, pass1 和 pass2, 功能由 ISP 的 hdr、3dnr、wdr、rgb2yuv 4 个组件组成。Pass1 是专门给 app 提供 raw buffer。

使能 AI ISP 时, 不支持 isp rotation/mirror/flip。

Pass0 和 pass2 有以下组合, 使用 [MI\\_ISP\\_CustSegAttr\\_t](#) 表示。

注意 pass0 模块序号不能比 pass2 大。

Pass0: HDR,HDR->3DNR,HDR->3DNR->WDR, HDR->3DNR->WDR->RGB2YUV。

Pass2:RGB2YUV,WDR->RGB2YUV,3DNR->WDR->RGB2YUV,HDR->3DNR->WDR->RGB2YUV

[E\\_MI\\_ISP\\_SEG\\_HDR](#) 输出是 before 3DNR。

[E\\_MI\\_ISP\\_SEG\\_3DNR](#) 包含 DNR Rotation/mirror/flip, 输出是 before WDR。

[E\\_MI\\_ISP\\_SEG\\_WDR](#) 输出是 before RGB2YUV。

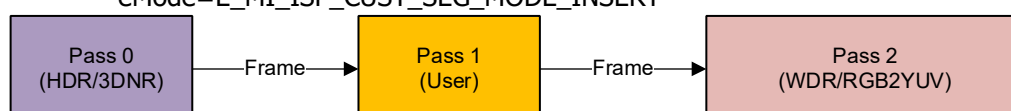
pstCustSegAttr->eMode、pstCustSegAttr->eFrom 和 pstCustSegAttr->eTo, 用于描述 pass1 的功能组成。

[E\\_MI\\_ISP\\_CUST\\_SEG\\_MODE\\_INSERT](#) 表示在 HDR->3DNR->WDR->RGB2YUV 中间某个地方插入 app 的图像处理 pass1。

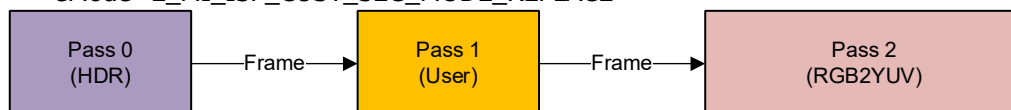
[E\\_MI\\_ISP\\_CUST\\_SEG\\_MODE\\_REPLACE](#), 表示替换 HDR、3DNR、WDR、RGB2YUV 的功能。可以只替换 3DNR, 只替换 WDR, 或 3DNR、WDR 都替换掉。

例如:

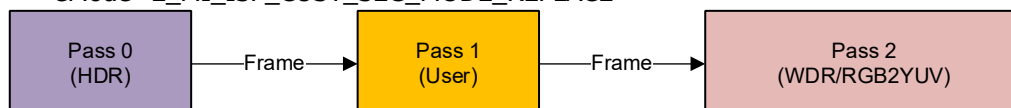
- eFrom= E\_MI\_ISP\_SEG\_3DNR,,eTo= E\_MI\_ISP\_SEG\_WDR,  
eMode=E\_MI\_ISP\_CUST\_SEG\_MODE\_INSERT



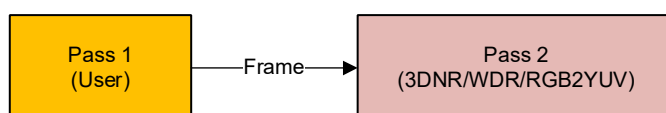
- b. eFrom= E\_MI\_ISP\_SEG\_3DNR,eTo= E\_MI\_ISP\_SEG\_WDR,  
eMode=E\_MI\_ISP\_CUST\_SEG\_MODE\_REPLACE



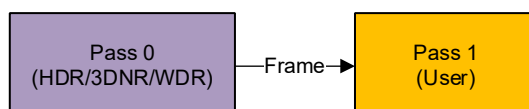
- c. eFrom= E\_MI\_ISP\_SEG\_3DNR,eTo= E\_MI\_ISP\_SEG\_3DNR,  
eMode=E\_MI\_ISP\_CUST\_SEG\_MODE\_REPLACE



- d. eFrom= E\_MI\_ISP\_SEG\_HDR,eTo= E\_MI\_ISP\_SEG\_HDR,  
eMode=E\_MI\_ISP\_CUST\_SEG\_MODE\_REPLACE



- e. eFrom= E\_MI\_ISP\_SEG\_RGB2YUV,eTo= E\_MI\_ISP\_SEG\_RGB2YUV,  
eMode=E\_MI\_ISP\_CUST\_SEG\_MODE\_REPLACE



MI\_ISP\_SetCustSegAttr 要在 MI\_ISP\_CreateChannel 后，在 MI\_ISP\_StartChannel 和 MI\_ISP\_SetOutputPortParam 前调用。

➤ 举例

MI\_ISP 初始化:

```

MI_S32 ST_IspModuleInit(MI_ISP_DEV IspDevId)
{
 MI_ISP_CHANNEL IspChnId = 0;
 MI_ISP_PORT IspOutPortId =1;

 MI_ISP_DevAttr t stCreateDevAttr;
 memset(&stCreateDevAttr, 0x0, sizeof(MI_ISP_DevAttr_t));
 stCreateDevAttr.u32DevStitchMask = E_MI_ISP_DEVICE_ID0;

 MI_ISP_CreateDevice(IspDevId, &stCreateDevAttr);

```

```

MI_ISP_ChannelAttr_t stIspChnAttr;
memset(&stIspChnAttr, 0x0, sizeof(MI_ISP_ChannelAttr_t));
stIspChnAttr.u32SensorBindId = E_MI_ISP_SENSOR0;
MI_ISP_CreateChannel(IspDevId, IspChnId, &stIspChnAttr);

MI_SYS_WindowRect_t stInputCropInfo;
stInputCropInfo.u16x=0;
stInputCropInfo.u16y=0;
stInputCropInfo.u16width=0;
stInputCropInfo.u16height=0;//width/height==0 no use crop.
MI_ISP_SetInputPortCrop(IspDevId, IspChnId, &stInputCropInfo);

MI_ISP_ChnParam_t stChnParam;
stChnParam.eHDRType = E_MI_ISP_HDR_TYPE_OFF;
stChnParam.e3DNRLevel = E_MI_ISP_3DNR_LEVEL2;
stChnParam.bMirror = FALSE;
stChnParam.bFlip = FALSE;
stChnParam.eRot = E_MI_SYS_ROTATE_NONE;
stChnParam.bY2bEnable = FALSE;
MI_ISP_SetChnParam(IspDevId, IspChnId, &stChnParam);

MI_ISP_CustSegAttr_t stCustSegAttr;
memset(&stCustSegAttr, 0x0, sizeof(MI_ISP_CustSegAttr_t));
stCustSegAttr.eMode = E_MI_ISP_CUST_SEG_MODE_INSERT;
stCustSegAttr.eFrom = E_MI_ISP_SEG_3DNR;
stCustSegAttr.eTo = E_MI_ISP_SEG_WDR;
stCustSegAttr.stInputParam.ePixelFormat =
RGB_BAYER_PIXEL(E_MI_SYS_DATA_PRECISION_12BPP, E_MI_SYS_PIXEL_BAYERID_GR);
stCustSegAttr.stOutputParam.ePixelFormat = stCustSegAttr.stInputParam.ePixelFormat;
MI_ISP_SetCustSegAttr(enDevId,enChnId, &stCustSegAttr);

MI_ISP_StartChannel(IspDevId, IspChnId);

MI_ISP_OutPortParam_t stIspOutputParam;
memset(&stIspOutputParam, 0x0, sizeof(MI_ISP_OutPortParam_t));
stIspOutputParam.stCropRect.u16x=0;
stIspOutputParam.stCropRect.u16y=0;
stIspOutputParam.stCropRect.u16Width=0;
stIspOutputParam.stCropRect.u16Height=0;//width/height use 0, not use crop
stIspOutputParam.ePixelFormat = E_MI_SYS_PIXEL_FRAME_YUV_SEMIPLANAR_YUV420;
MI_ISP_SetOutputPortParam(IspDevId, IspChnId, IspOutPortId, &stIspOutputParam);
MI_ISP_EnableOutputPort(IspDevId, IspChnId, IspOutPortId);

return MI_SUCCESS;

```

```
}

```

获取 raw

•

```
MI_SYS_BUF_HANDLE hBufHandle = NULL;
MI_SYS_BufInfo_t stInputBufInfo;
MI_SYS_BufInfo_t stOutputBufInfo;
MI_U8 *pSrcData = NULL, *pDstData = NULL;

memset(&stInputBufInfo, 0, sizeof(MI_SYS_BufInfo_t));
memset(&stOutputBufInfo, 0, sizeof(MI_SYS_BufInfo_t));
s32Ret = MI_ISP_GetCustSegBuf(stChnPort.u32DevId, stChnPort.u32ChnId,
&hBufHandle,
 &stInputBufInfo, &stOutputBufInfo, 200);
if(s32Ret == MI_SUCCESS)
{
 pSrcData = stInputBufInfo.stFrameData->pVirAddr[0];
 pDstData = stOutputBufInfo.stFrameData->pVirAddr[0];
 memcpy(pDstData, pSrcData, stOutputBufInfo.stFrameData.u32BufSize);
 memset(pDstData, 0, stOutputBufInfo.stFrameData.u32BufSize/4);
 MI_ISP_PutCustSegBuf(stChnPort.u32DevId, stChnPort.u32ChnId, hBufHandle);
}
```

- 相关主题  
无。

2.25. MI\_ISP\_GetCustSegAttr

- 描述  
获取 AI ISP 属性。

- 语法  
MI\_S32 MI\_ISP\_GetCustSegAttr(MI\_ISP\_DEV DevId, MI\_ISP\_CHANNEL ChnId, MI\_ISP\_CustSegAttr\_t \*pstCustSegAttr);

- 形参

| 参数名称           | 描述       | 输入/输出 |
|----------------|----------|-------|
| DevId          | ISP 设备号。 | 输入    |
| ChnId          | ISP 通道号  | 输入    |
| pstCustSegAttr | AI ISP属性 | 输出    |

- 返回值  
MI\_SUCCESS (0) 成功。

返回值

非 0 失败，详情参照[错误码](#)。

- 依赖
  - 头文件：mi\_isp\_datatype.h、mi\_isp.h
  - 库文件：libmi\_isp.a

※ 注意  
无。

➤ 举例  
无。

➤ 相关主题  
无。

2.26. MI\_ISP\_GetCustSegBuf

- 描述  
获取 AI ISP buffer。
- 语法  
MI\_S32 MI\_ISP\_GetCustSegBuf(MI\_ISP\_DEV DevId, MI\_ISP\_CHANNEL ChnId,MI\_SYS\_BUF\_HANDLE \*pBufHandle, MI\_SYS\_BufInfo\_t \*pstInputBufInfo, MI\_SYS\_BufInfo\_t \*pstOutputBufInfo, MI\_S32 s32MilliSec);

➤ 形参

| 参数名称             | 描述                              | 输入/输出 |
|------------------|---------------------------------|-------|
| DevId            | ISP 设备号。                        | 输入    |
| ChnId            | ISP 通道号                         | 输入    |
| pBufHandle,      | Pass1 input/output Buf的handle句柄 | 输出    |
| pstInputBufInfo  | Pass1 input buffer              | 输出    |
| pstOutputBufInfo | Pass1 output buffer             | 输出    |
| s32MilliSec      | 超时时间 ms                         | 输入    |

- 返回值

返回值

MI\_SUCCESS（0）成功。

非 0 失败，详情参照[错误码](#)。

➤ 依赖

- 头文件：mi\_isp\_datatype.h、mi\_isp.h
- 库文件：libmi\_isp.a

※ 注意

和 [MI\\_ISP\\_PutCustSegBuf](#) 成对使用。

➤ 举例

参考 [MI\\_ISP\\_SetCustSegAttr](#) 举例。

➤ 相关主题

无。

## 2.27. MI\_ISP\_PutCustSegBuf

➤ 描述

释放 AI ISP buffer。

➤ 语法

```
MI_S32 MI_ISP_PutCustSegBuf(MI_ISP_DEV DevId, MI_ISP_CHANNEL ChnId,
MI_SYS_BUF_HANDLE bufHandle);
```

➤ 形参

| 参数名称        | 描述                              | 输入/输出 |
|-------------|---------------------------------|-------|
| DevId       | ISP 设备号。                        | 输入    |
| ChnId       | ISP 通道号                         | 输入    |
| pBufHandle, | Pass1 input/output Buf的handle句柄 | 输入    |

➤ 返回值

返回值 { MI\_SUCCESS (0) 成功。  
非 0 失败，详情参照[错误码](#)。

- 依赖
- 头文件：mi\_isp\_datatype.h、mi\_isp.h
  - 库文件：libmi\_isp.a

※ 注意

- 和 [MI\\_ISP\\_GetCustSegBuf](#) 成对使用。
- 要按 [MI\\_ISP\\_GetCustSegBuf](#) 获取 buffer 的顺序，调用 [MI\\_ISP\\_PutCustSegBuf](#)，不能乱序 put buffer。

➤ 举例

参考 [MI\\_ISP\\_SetCustSegAttr](#) 举例。



- 相关主题  
无。

## 2.28. MI\_ISP\_GetSubChnId

- 描述

获取 ISP 的子通道号。

- 语法

```
MI_S32 MI_ISP_GetSubChnId(MI_ISP_DEV DevId, MI_ISP_CHANNEL MainChnId,
MI_ISP_BindSnrId_e eSensorBindId, MI_U32 *pu32SubChnId);
```

- 形参

| 参数名称          | 描述            | 输入/输出 |
|---------------|---------------|-------|
| DevId         | ISP 设备号。      | 输入    |
| MainChnId     | ISP 通道号       | 输入    |
| eSensorBindId | 子通道绑定的sensor号 | 输入    |
| pu32SubChnId  | ISP 子通道号      | 输出    |

- 返回值

返回值 { MI\_SUCCESS (0) 成功。  
非 0 失败，详情参照[错误码](#)。

- 依赖

- 头文件: mi\_isp\_datatype.h、mi\_isp.h
- 库文件: libmi\_isp.a

- ※ 注意

- 在 Stitch 场景下，一个 MI\_ISP Channel 会绑定多个 Sensor，当 IQ 要控制对应 Sensor 时，需要输入 MI\_ISP DevId，MI\_ISP ChannelId 和 SensorId，获取对应的 SubChannelId，也就是 IQ 所使用的通道。
- 其他情况下 IQ 所使用的通道和 MI\_ISP ChannelId 一致。

- 举例

无

- 相关主题  
无。

### 3. ISP 数据类型

视频输入相关数据类型定义如下：

表 3-3: ISP 数据类型

| 数据类型                                         | 描述                     |
|----------------------------------------------|------------------------|
| <a href="#">MI_ISP_DEV</a>                   | 定义 ISP 设备的类型           |
| <a href="#">MI_ISP_CHANNEL</a>               | 定义 ISP 通道的类型           |
| <a href="#">MI_ISP_PORT</a>                  | 定义 ISP 端口的类型           |
| <a href="#">MI_ISP_DevMaskId_e</a>           | 定义 ISP 设备需要mask使用的设备ID |
| <a href="#">MI_ISP_DevAttr_t</a>             | 定义 ISP 设备属性参数          |
| <a href="#">MI_ISP_HDRType_e</a>             | 定义 HDR 开关模式            |
| <a href="#">MI_ISP_3DNR_Level_e</a>          | 定义 3DNR 设置等级           |
| <a href="#">MI_ISP_BindSnrId_e</a>           | 定义 ISP 绑定 sensor 的 ID  |
| <a href="#">MI_ISP_SYNC3A_e</a>              | 定义同步各通道 3A 参数          |
| <a href="#">MI_ISP_IQApiHeader_t</a>         | 定义IQ api 数据类型          |
| <a href="#">MI_ISP_VersionPara_t</a>         | 定义ISP特殊参数版本            |
| <a href="#">MI_ISP_CustIQParam_t</a>         | 定义ISP初始化参数             |
| <a href="#">MI_ISP_ChannelAttr_t</a>         | 定义 ISP 通道静态属性参数        |
| <a href="#">MI_ISP_ChnParam_t</a>            | 定义 ISP 通道动态属性参数        |
| <a href="#">MI_ISP_OutPortParam_t</a>        | 定义 ISP 输出端口的参数         |
| <a href="#">MI_ISP_CALLBK_FUNC</a>           | 定义回调函数类型               |
| <a href="#">MI_ISP_CallbackMode_e</a>        | 定义回调模式                 |
| <a href="#">MI_ISP_IrqType_e</a>             | 定义硬件中断类型               |
| <a href="#">MI_ISP_CallbackParam_t</a>       | 定义回调参数                 |
| <a href="#">MI_ISP_ZoomEntry_t</a>           | 定义 Zoom 单元条目类型         |
| <a href="#">MI_ISP_ZoomTable_t</a>           | 定义 Zoom Table 类型       |
| <a href="#">MI_ISP_ZoomAttr_t</a>            | 定义 Zoom 属性             |
| <a href="#">MI_ISP_CustSegAttr_t</a>         | 定义AI ISP属性             |
| <a href="#">MI_ISP_CustSegInPortParam_t</a>  | Pass1 input port的属性    |
| <a href="#">MI_ISP_CustSegOutPortParam_t</a> | Pass1 output port的属性   |
| <a href="#">MI_ISP_CustSegMode_e</a>         | AI ISP的模式              |
| <a href="#">MI_ISP_InternalSeg_e</a>         | ISP的模块名称               |

### 3.1. MI\_ISP\_DEV

- 说明  
定义 ISP 设备的类型。
- 定义  
`typedef MI_U32 MI_ISP_DEV;`
- ※ 注意事项  
无。
- 相关数据类型及接口  
无。

### 3.2. MI\_ISP\_CHANNEL

- 说明  
定义 ISP 通道的类型。
- 定义  
`typedef MI_U32 MI_ISP_CHANNEL;`
- ※ 注意事项  
无。
- 相关数据类型及接口  
无。

### 3.3. MI\_ISP\_PORT

- 说明  
定义 ISP 端口的类型。
- 定义  
`typedef MI_U32 MI_ISP_PORT;`
- ※ 注意事项  
无。
- 相关数据类型及接口  
无。

### 3.4. MI\_ISP\_DevMaskId\_e

➤ 说明

定义 ISP 设备需要 mask 使用的设备 ID。

➤ 定义

```
typedef enum
{
 E_MI_ISP_DEVICEMASK_ID0 = 0x0001,
 E_MI_ISP_DEVICEMASK_ID1 = 0x0002,
 E_MI_ISP_DEVICEMASK_ID_MAX = 0xffff
} MI_ISP_DevMaskId_e;
```

※ 注意事项

Tiramisu / Mochi/ Maruko 系列芯片只有 ISP E\_MI\_ISP\_DEVICEMASK\_ID0 使用。

➤ 相关数据类型及接口

[MI\\_ISP\\_CreateDevice](#)

### 3.5. MI\_ISP\_DevAttr\_t

➤ 说明

定义 ISP 设备属性参数。

➤ 定义

```
typedef struct MI_ISP_DevAttr_s
{
 MI_U32 u32DevStitchMask; //multi ISP dev bitmask by MI_ISP_DevMaskId_e
}MI_ISP_DevAttr_t;
```

➤ 成员

| 成员名称             | 描述             |
|------------------|----------------|
| u32DevStitchMask | ISP DEV ID 掩码。 |

※ 注意事项

u32DevStitchMask 由 MI\_ISP\_DevMaskId\_e, bitmask 组成。如果通道上的数据量一个 isp device 处理速度无法达到要求，可以通过该参数复用另外一个 Device。复用之后不可以再创建另外一个 Device。

➤ 相关数据类型及接口

[MI\\_ISP\\_CreateDevice](#)

### 3.6. MI\_ISP\_HDRTType\_e

➤ 说明

定义 HDR 开关模式

➤ 定义

```
typedef enum
{
 E_MI_ISP_HDR_TYPE_OFF,
 E_MI_ISP_HDR_TYPE_VC,
 E_MI_ISP_HDR_TYPE_DOL,
 E_MI_ISP_HDR_TYPE_EMBEDDED,
 E_MI_ISP_HDR_TYPE_LI,
 E_MI_ISP_HDR_TYPE_MAX
} MI_ISP_HDRTType_e;
```

➤ 成员

| 成员名称                       | 描述                                                |
|----------------------------|---------------------------------------------------|
| E_MI_ISP_HDR_TYPE_OFF      | 不开HDR                                             |
| E_MI_ISP_HDR_TYPE_VC       | virtual channel mode HDR,<br>vc0->long,vc1->short |
| E_MI_ISP_HDR_TYPE_DOL      | Digital Overlap High Dynamic Range                |
| E_MI_ISP_HDR_TYPE_EMBEDDED | compressed HDR mode                               |
| E_MI_ISP_HDR_TYPE_LI       | Line interlace HDR                                |
| E_MI_ISP_HDR_TYPE_MAX      | HDR 边界值                                           |

※ 注意事项

具体使用的 HDR Type 可以通过 MI\_SNR\_GetPadInfo 接口获取。  
HDR 功能需要 MI\_VIF 和 MI\_ISP 模块配合完成，需要两边设置相同的 HDR 类型。

➤ 相关数据类型及接口

[MI\\_ISP\\_ChnParam\\_t](#)

### 3.7. MI\_ISP\_3DNR\_Level\_e

➤ 说明

定义 3DNR 设置等级。

➤ 定义

```
typedef enum
{
 E_MI_ISP_3DNR_LEVEL_OFF,
 E_MI_ISP_3DNR_LEVEL1,
 E_MI_ISP_3DNR_LEVEL2,
```

```
E_MI_ISP_3DNR_LEVEL3,
E_MI_ISP_3DNR_LEVEL4,
E_MI_ISP_3DNR_LEVEL5,
E_MI_ISP_3DNR_LEVEL6,
E_MI_ISP_3DNR_LEVEL7,
E_MI_ISP_3DNR_LEVEL_NUM
}MI_ISP_3DNR_Level_e;
```

※ 注意事项

3DNR level 越高，降噪效果越好，消耗的 buffer 也越多。

| 芯片       | 3DNR支持的最大等级          |
|----------|----------------------|
| Tiramisu | E_MI_ISP_3DNR_LEVEL2 |
| Muffin   | E_MI_ISP_3DNR_LEVEL2 |
| Mochi    | E_MI_ISP_3DNR_LEVEL7 |
| Maruko   | E_MI_ISP_3DNR_LEVEL7 |

➤ 相关数据类型及接口

[MI\\_ISP\\_ChnParam\\_t](#)

## 3.8. MI\_ISP\_BindSnrId\_e

➤ 说明

定义 ISP 绑定 sensor 的 ID。

➤ 定义

```
typedef enum
{
 E_MI_ISP_SENSOR_INVALID = 0,
 E_MI_ISP_SENSOR0 = 0x1,
 E_MI_ISP_SENSOR1 = 0x2,
 E_MI_ISP_SENSOR2 = 0x4,
 E_MI_ISP_SENSOR3 = 0x8,
 E_MI_ISP_SENSOR4 = 0x10,
 E_MI_ISP_SENSOR5 = 0x20,
 E_MI_ISP_SENSOR6 = 0x40,
 E_MI_ISP_SENSOR7 = 0x80,
 E_MI_ISP_SENSOR_MAX = 8
}MI_ISP_BindSnrId_e;
```

※ 注意事项

无。

➤ 相关数据类型及接口

无。



### 3.9. MI\_ISP\_SYNC3A\_e

➤ 说明

定义同步各通道 3A 参数。

➤ 定义

```
typedef enum
{
 E_MI_ISP_SYNC3A_NONE = 0x00,
 E_MI_ISP_SYNC3A_AE = 0x01,
 E_MI_ISP_SYNC3A_AWB = 0x02,
 E_MI_ISP_SYNC3A_IQ = 0x04
}MI_ISP_SYNC3A_e;
```

➤ 成员

| 成员名称                 | 描述         |
|----------------------|------------|
| E_MI_ISP_SYNC3A_NONE | 不同步 3A     |
| E_MI_ISP_SYNC3A_AE   | 同步自动曝光     |
| E_MI_ISP_SYNC3A_AWB  | 同步自动白平衡    |
| E_MI_ISP_SYNC3A_IQ   | 同步图像质量参数设置 |

※ 注意事项

无。

➤ 相关数据类型及接口

无。

### 3.10. MI\_ISP\_IQApiHeader\_t

➤ 说明

定义设定 IQ api 数据属性。

➤ 定义

```
typedef struct MI_ISP_IQApiHeader_s
{
 MI_U32 u32HeadSize; //Size of MIISPApiHeader_t
 MI_U32 u32DataLen; //Data length;
 MI_U32 u32CtrlID; //Function ID
 MI_U32 u32Channel; //Isp channel number
 MI_U32 u32DevId; //Isp Device Id
 MI_S32 s32Ret; //Isp api retuen value
} MI_ISP_IQApiHeader_t;
```

➤ 成员

| 成员名称        | 描述                               |
|-------------|----------------------------------|
| u32HeadSize | MI_ISP_IQApiHeader_t struct 占用空间 |



| 成员名称       | 描述              |
|------------|-----------------|
| u32DataLen | 数据 size         |
| u32CtrlID  | 功能对应 control Id |
| u32Channel | Isp 通道 Id       |
| u32DevId   | Isp 设备 Id       |
| s32Ret     | 接口返回值           |

※ 注意事项  
无。

➤ 相关数据类型及接口  
无。

### 3.11. MI\_ISP\_VersionPara\_t

➤ 说明

定义 ISP 特殊参数版本。

➤ 定义

```
typedef struct MI_ISP_VersionPara_s
{
 MI_U32 u32Revision;
 MI_U32 u32Size;
 MI_U8 u8Data[VERSIONPARA_DATA_SIZE];
}MI_ISP_VersionPara_t;
```

➤ 成员

| 成员名称                          | 描述                              |
|-------------------------------|---------------------------------|
| u32Revision                   | 和IQ team 协商定义的对版本号              |
| u32Size                       | 数据对应有效size                      |
| u8Data[VERSIONPARA_DATA_SIZE] | 存储数据，VERSIONPARA_DATA_SIZE的值是64 |

※ 注意事项  
无。

➤ 相关数据类型及接口  
[MI\\_ISP\\_CustIQParam\\_t](#)

### 3.12. MI\_ISP\_CustIQParam\_t

➤ 说明

定义 ISP IQ 初始化参数。

➤ 定义

```
typedef struct MI_ISP_CustIQParam_s
{
 MI_ISP_VersionPara_t stVersion;
}MI_ISP_CustIQParam_t;
```

➤ 成员

| 成员名称      | 描述   |
|-----------|------|
| stVersion | 版本参数 |

※ 注意事项

需要 ISP 客制化处理的一些 IQ 初始化参数，可以通过该参数设置，如果没有需求，可以都设置 0。

➤ 相关数据类型及接口

无。

### 3.13. [MI\\_ISP\\_ChannelAttr\\_t](#)

➤ 说明

定义 ISP 通道静态属性参数。

➤ 定义

```
typedef struct MI_ISP_ChannelAttr_s
{
 MI_U32 u32SensorBindId; //bitmask by MI_ISP_BindSnrId_e
 MI_ISP_CustIQParam_t stIspCustIqParam;
 MI_U32 u32Sync3AType; //sync 3a bitmask by MI_ISP_SYNC3A_e
}MI_ISP_ChannelAttr_t;
```

➤ 成员

| 成员名称             | 描述                   |
|------------------|----------------------|
| u32SensorBindId  | 通道绑定对应的sensor Id     |
| stIspCustIqParam | Isp IQ初始化参数          |
| u32Sync3AType    | 同步通道内多组图像之间的3A/IQ 参数 |

※ 注意事项

- 当多 sensor 拼接场景一个 ISP channel 需要处理多个 sensor 图像，需要通过 u32SensorBindId 将多个 sensor Id bitmask 起来设置，否则只设置对应 sensor Id 接口，如果不需要 ISP 做 IQ/3A，或者前端数据源不是 sensor，可以设置 0。
- 当多 sensor 拼接场景一个 ISP channel 需要处理多个 sensor 图像，通过 u32Sync3AType 设置多个图像之间 3A/IQ 效果同步，避免两个图像之间效果差别过大。

➤ 相关数据类型及接口

[MI\\_ISP\\_CreateChannel](#)

### 3.14. MI\_ISP\_ChnParam\_t

➤ 说明

定义 ISP 通道动态属性参数。

➤ 定义

```
typedef struct MI_ISP_ChnParam_s
{
 MI_ISP_HDRTType_e eHDRTType;
 MI_ISP_3DNR_Level_e e3DNRLevel;
 MI_BOOL bMirror;
 MI_BOOL bFlip;
 MI_SYS_Rotate_e eRot;
 MI_BOOL bY2bEnable;
} MI_ISP_ChnParam_t;
```

➤ 成员

| 成员名称       | 描述                  |
|------------|---------------------|
| eHDRTType  | HDR 开关参数            |
| e3DNRLevel | 3dnr 量级参数           |
| bMirror    | 使能通道Mirror          |
| bFlip      | 使能通道Flip            |
| eRot       | 通道rotation参数        |
| bY2bEnable | 使能yuv to bayer的转换功能 |

※ 注意事项

- 非 Maruko 的 chip, eRot>0 或者 bFlip=True 时, e3DNRLevel 必须大于 0。
- bY2bEnable 只有 Muffin 支持, 且只有 input port 是 Frame Mode 时支持。
- Mochi 不支持 HDR/Mirror/Flip/Rot/Y2bEnable 功能。

➤ 相关数据类型及接口

[MI\\_ISP\\_SetChnParam](#)

### 3.15. MI\_ISP\_OutPortParam\_t

➤ 说明

定义 ISP 输出端口的参数。

➤ 定义

```
typedef struct MI_ISP_OutPortParam_s
{
 MI_SYS_WindowRect_t stCropRect;
 MI_SYS_PixelFormat_e ePixelFormat;
 MI_SYS_CompressMode_e eCompressMode;
}MI_ISP_OutPortParam_t;
```

➤ 成员

| 成员名称          | 描述       |
|---------------|----------|
| stCropRect    | 输出裁剪区域   |
| ePixelFormat  | 像素格式     |
| eCompressMode | 输出像素压缩模式 |

※ 注意事项

- eCompressMode 支持范围

| 芯片       | 支持的输出像素压缩模式                                                   |
|----------|---------------------------------------------------------------|
| Tiramisu | E_MI_SYS_COMPRESS_MODE_NONE                                   |
| Muffin   | E_MI_SYS_COMPRESS_MODE_NONE                                   |
| Mochi    | E_MI_SYS_COMPRESS_MODE_NONE<br>E_MI_SYS_COMPRESS_MODE_TO_6BIT |
| Maruko   | E_MI_SYS_COMPRESS_MODE_TO_6BIT                                |

➤ 相关数据类型及接口

无。

## 3.16. MI\_ISP\_CALLBK\_FUNC

➤ 说明

定义回调函数类型。

➤ 定义

```
typedef MI_S32 (*MI_ISP_CALLBK_FUNC)(MI_U64 u64Data);
```

※ 注意事项

无。

➤ 相关数据类型及接口

[MI\\_ISP\\_CallbackParam\\_t](#)

## 3.17. MI\_ISP\_CallbackMode\_e

➤ 说明

定义回调模式。

➤ 定义

```
typedef enum
{
 E_MI_ISP_CALLBACK_ISR,
 E_MI_ISP_CALLBACK_MAX,
} MI_ISP_CallbackMode_e;
```

➤ 成员

| 成员名称                  | 描述       |
|-----------------------|----------|
| E_MI_ISP_CALLBACK_ISR | 硬件中断模式回调 |
| E_MI_ISP_CALLBACK_MAX | 回调模式最大值  |

※ 注意事项

目前只支持 ISR 回调模式。

➤ 相关数据类型及接口

[MI\\_ISP\\_CallbackParam\\_t](#)

## 3.18. MI\_ISP\_IrqType\_e

➤ 说明

定义硬件中断类型。

➤ 定义

```
typedef enum
{
 E_MI_ISP_IRQ_ISPVSYNCR,
 E_MI_ISP_IRQ_ISPFRAMEDONE,
 E_MI_ISP_IRQ_MAX,
} MI_ISP_IrqType_e;
```

➤ 成员

| 成员名称                      | 描述                  |
|---------------------------|---------------------|
| E_MI_ISP_IRQ_ISPVSYNCR    | ISP Vsync 中断类型      |
| E_MI_ISP_IRQ_ISPFRAMEDONE | ISP Frame done 中断类型 |
| E_MI_ISP_IRQ_MAX          | ISP 硬件中断类型最大值       |

※ 注意事项

- E\_MI\_VPE\_IRQ\_ISPVSYNCR: 每一帧第一个 pixel 的信号
- E\_MI\_VPE\_IRQ\_ISPFRAMEDONE: isp 写出每一张结束的信号。

➤ 相关数据类型及接口

[MI\\_ISP\\_CallbackParam\\_t](#)

## 3.19. MI\_ISP\_CallbackParam\_t

➤ 说明

定义回调参数。

➤ 定义

```
typedef struct MI_ISP_CallbackParam_s
{
```

```

MI_ISP_CallbackMode_e eCallbackMode;
MI_ISP_IrqType_e eIrqType;
MI_ISP_CALLBACK_FUNC pfnCallbackFunc;
MI_U64 u64Data;
} MI_ISP_CallbackParam_t;

```

➤ 成员

| 成员名称            | 描述       |
|-----------------|----------|
| eCallbackMode   | 回调模式     |
| eIrqType        | 硬件中断模式类型 |
| pfnCallbackFunc | 回调接口指针   |
| u64Data         | 回调接口参数   |

※ 注意事项

无。

➤ 相关数据类型及接口

[MI\\_ISP\\_CallbackTask\\_Register](#)  
[MI\\_ISP\\_CallbackTask\\_Unregister](#)

## 3.20. MI\_ISP\_ZoomEntry\_t

➤ 说明

定义 Zoom 单元条目类型。

➤ 定义

```

typedef struct MI_ISP_ZoomEntry_s
{
 MI_SYS_WindowRect_t stCropWin;
 MI_U8 u8ZoomSensorId;
} MI_ISP_ZoomEntry_t;

```

➤ 成员

| 成员名称           | 描述                   |
|----------------|----------------------|
| stCropWin      | Crop 位置参数            |
| u8ZoomSensorId | Crop 参数对应的 sensor Id |

※ 注意事项

u8ZoomSensorId 和 sensor driver 中 pCus\_sensor\_GetCurSwitchSensorId 回调函数回传值对应。

➤ 相关数据类型及接口

[MI\\_ISP\\_ZoomTable\\_t](#)

### 3.21. MI\_ISP\_ZoomTable\_t

➤ 说明

定义 Zoom Table 类型。

➤ 定义

```
typedef struct MI_ISP_ZoomTable_s
{
 MI_U32 u32EntryNum;
 MI_ISP_ZoomEntry_t *pVirTableAddr;
} MI_ISP_ZoomTable_t;
```

➤ 成员

| 成员名称          | 描述                        |
|---------------|---------------------------|
| u32EntryNum   | Zoom Table 中包含条目数量        |
| pVirTableAddr | Zoom Table Buffer Pointer |

※ 注意事项

无。

➤ 相关数据类型及接口

[MI\\_ISP\\_LoadPortZoomTable](#)

### 3.22. MI\_ISP\_ZoomAttr\_t

➤ 说明

定义 Zoom 属性。

➤ 定义

```
typedef struct MI_ISP_ZoomAttr_s
{
 MI_U32 u32FromEntryIndex;
 MI_U32 u32ToEntryIndex;
 MI_U32 u32CurEntryIndex;
} MI_ISP_ZoomAttr_t;
```

➤ 成员

| 成员名称              | 描述              |
|-------------------|-----------------|
| u32FromEntryIndex | 开始运行 Zoom 的条目索引 |
| u32ToEntryIndex   | 结束运行 Zoom 的条目索引 |
| u32CurEntryIndex  | 当前 Zoom 位置的索引   |

※ 注意事项

无。

➤ 相关数据类型及接口

[MI\\_ISP\\_StartPortZoom](#)  
[MI\\_ISP\\_GetPortCurZoomAttr](#)

### 3.23. MI\_ISP\_CustSegAttr\_t

➤ 说明

定义 AI ISP 属性。

➤ 定义

```
typedef struct MI_ISP_CustSegAttr_s
{
 MI_ISP_CustSegMode_e eMode;
 MI_ISP_InternalSeg_e eFrom;
 MI_ISP_InternalSeg_e eTo;
 MI_ISP_CustSegInPortParam_t stInputParam;
 MI_ISP_CustSegOutPortParam_t stOutputParam;
} MI_ISP_CustSegAttr_t;
```

➤ 成员

| 成员名称          | 描述                    |
|---------------|-----------------------|
| eMode         | Pass1 的模式             |
| eFrom         | Pass1 的起始模块           |
| eTo           | Pass1 的结束模块           |
| stInputParam  | Pass1 input port 的属性  |
| stOutputParam | Pass1 output port 的属性 |

※ 注意事项

参数具体作用，请参考 [MI\\_ISP\\_SetCustSegAttr](#) 的说明。

➤ 相关数据类型及接口

[MI\\_ISP\\_SetCustSegAttr](#)  
[MI\\_ISP\\_GetCustSegAttr](#)

### 3.24. MI\_ISP\_CustSegInPortParam\_t

➤ 说明

Pass1 input port 的属性。

➤ 定义

```
typedef struct MI_ISP_CustSegInPortParam_s
{
 MI_SYS_PixelFormat_e ePixelFormat;
} MI_ISP_CustSegInPortParam_t;
```



➤ 成员

| 成员名称         | 描述   |
|--------------|------|
| ePixelFormat | 数据格式 |

※ 注意事项

Before 3DNR 只支持 12bit bayer 和 16bit bayer 的输入输出。

Before WDR 只支持 16bit bayer 的输入输出。

Before RGB2YUV 只支持 E\_MI\_SYS\_PIXEL\_FRAME\_ARGB8888 和 E\_MI\_SYS\_PIXEL\_FRAME\_RGB101010 的输入输出。

➤ 相关数据类型及接口

[MI\\_ISP\\_SetCustSegAttr](#)

[MI\\_ISP\\_GetCustSegAttr](#)

## 3.25. MI\_ISP\_CustSegOutPortParam\_t

➤ 说明

Pass1 output port 的属性。

➤ 定义

```
typedef struct MI_ISP_CustSegOutPortParam_s
{
 MI_SYS_PixelFormat_e ePixelFormat;
} MI_ISP_CustSegOutPortParam_t;
```

➤ 成员

| 成员名称         | 描述   |
|--------------|------|
| ePixelFormat | 数据格式 |

※ 注意事项

• Before 3DNR 只支持 12bit bayer 和 16bit bayer 的输入输出。

• Before WDR 只支持 16bit bayer 的输入输出。

• Before RGB2YUV 只支持 E\_MI\_SYS\_PIXEL\_FRAME\_ARGB8888 和 E\_MI\_SYS\_PIXEL\_FRAME\_RGB101010 的输入输出。

➤ 相关数据类型及接口

[MI\\_ISP\\_SetCustSegAttr](#)

[MI\\_ISP\\_GetCustSegAttr](#)

## 3.26. MI\_ISP\_CustSegMode\_e

➤ 说明

AI ISP 的模式。

➤ 定义

```
typedef enum
{
 E_MI_ISP_CUST_SEG_MODE_NONE = 0,
 E_MI_ISP_CUST_SEG_MODE_INSERT,
 E_MI_ISP_CUST_SEG_MODE_REPLACE,
 E_MI_ISP_CUST_SEG_MODE_MAX
} MI_ISP_CustSegMode_e;
```

➤ 成员

| 成员名称                           | 描述        |
|--------------------------------|-----------|
| E_MI_ISP_CUST_SEG_MODE_NONE    | 关闭 AI ISP |
| E_MI_ISP_CUST_SEG_MODE_INSERT  | 插入模式      |
| E_MI_ISP_CUST_SEG_MODE_REPLACE | 替换模式      |
| E_MI_ISP_CUST_SEG_MODE_MAX     | 模式的最大值    |

※ 注意事项

无。

➤ 相关数据类型及接口

[MI\\_ISP\\_SetCustSegAttr](#)

[MI\\_ISP\\_GetCustSegAttr](#)

## 3.27. MI\_ISP\_InternalSeg\_e

➤ 说明

ISP 的模块名称。

➤ 定义

```
typedef enum
{
 E_MI_ISP_SEG_INVALID = 0,
 E_MI_ISP_SEG_HDR, // HDR
 E_MI_ISP_SEG_3DNR, // 3DNR
 E_MI_ISP_SEG_WDR, // WDR
 E_MI_ISP_SEG_RGB2YUV, // RGB2YUV
 E_MI_ISP_SEG_MAX
} MI_ISP_InternalSeg_e;
```

➤ 成员

| 成员名称                 | 描述                          |
|----------------------|-----------------------------|
| E_MI_ISP_SEG_INVALID | 无效模块                        |
| E_MI_ISP_SEG_HDR     | HDR 模块                      |
| E_MI_ISP_SEG_3DNR    | DNR,Rotation/mirror/flip 模块 |
| E_MI_ISP_SEG_WDR     | WDR 模块                      |
| E_MI_ISP_SEG_RGB2YUV | RGB2YUV 模块                  |

|                  |        |
|------------------|--------|
| E_MI_ISP_SEG_MAX | 模块的最大值 |
|------------------|--------|

※ 注意事项  
无。

➤ 相关数据类型及接口

[MI\\_ISP\\_SetCustSegAttr](#)

[MI\\_ISP\\_GetCustSegAttr](#)

## 4. ISP 错误码

---

API 错误码如下表所示。

表 4-4: API 错误码

| 错误代码       | 宏定义                       | 描述       |
|------------|---------------------------|----------|
| 0xA0078001 | MI_ERR_ISP_INVALID_CHNID  | 设备通道号无效  |
| 0xA0078002 | MI_ERR_ISP_INVALID_PORTID | 设备端口号无效  |
| 0xA0078003 | MI_ERR_ISP_ILLEGAL_PARAM  | 非法参数     |
| 0xA0078004 | MI_ERR_ISP_EXIST          | 设备已经退出   |
| 0xA0078005 | MI_ERR_ISP_UNEXIST        | 设备还没有退出  |
| 0xA0078006 | MI_ERR_ISP_NULL_PTR       | 空指针参数    |
| 0xA0078008 | MI_ERR_ISP_NOT_SUPPORT    | 不支持      |
| 0xA0078009 | MI_ERR_ISP_NOT_PERM       | 操作不允许    |
| 0xA007800C | MI_ERR_ISP_NOMEM          | 分配内存失败   |
| 0xA007800D | MI_ERR_ISP_NOBUF          | 内存已经使用完  |
| 0xA007800E | MI_ERR_ISP_BUF_EMPTY      | 视频输入缓存为空 |
| 0xA0078010 | MI_ERR_ISP_NOTREADY       | 设备未初始化完成 |
| 0xA0078012 | MI_ERR_ISP_BUSY           | 设备系统繁忙   |

## 5. PROCFS 介绍

### 5.1. cat

- 调试信息

# cat /proc/mi\_modules/mi\_isp/mi\_isp0

```
----- end dump ISP Dev info -----
DevID CreChnNum Devmask cmdq En Mode num VsyncCnt FrameDoneCnt DropCnt
0 2 1 c662b788 1 1 27 370 371 0
----- End dump ISP dev 0 info -----

----- start dump ISP CHN info -----
DevId ChnId start SnrId sync3A chnNum Crop InSize pixel Stride Atom Atom0
0 0 1 0 0 1(0, 0,1280, 720)(1280, 720)YUV422UYVY 0 0 185
0 1 1 0 0 1(0, 0,1280, 720)(1280, 720)YUV422UYVY 0 0 185

DevId ChnId Rot bMirr/flip 3DNRLevel HdrMode
0 0 0(0, 0) 2 0
0 1 0(0, 0) 2 0

DevId ChnId PreCnt EnqCnt BarCnt checkin checkout DeqCnt DropCnt EnqOTNull
0 0 185 185 185 185 185 185 5 0
0 1 185 185 185 185 185 185 4 0
----- End dump ISP CHN info -----

----- start dump ISP OUTPUT PORT info -----
DevId ChnId PortId bindtype Enable Pixel PortCrop Stride GetCnt FailCnt FinishCnt fps
0 0 0 Real 1YUV422UYVY(0, 0,1280, 720) 0 185 0 0 185 25.00
0 0 1 Frame 0YUV422UYVY(0, 0, 0, 0) 0 0 0 0 0 0.00
0 0 2 Frame 0YUV422UYVY(0, 0, 0, 0) 0 0 0 0 0 0.00
0 1 0 Real 1YUV422UYVY(0, 0,1280, 720) 0 185 0 0 185 25.00
0 1 1 Frame 0YUV422UYVY(0, 0, 0, 0) 0 0 0 0 0 0.00
0 1 2 Frame 0YUV422UYVY(0, 0, 0, 0) 0 0 0 0 0 0.00
----- End dump ISP OUTPUT PORT info -----
```

- 调试信息分析

记录当前 ISP 的使用状况以及相关属性，可以动态地获取到这些信息，方便调试和测试。

- 参数说明

| 参数           |           | 描述              |
|--------------|-----------|-----------------|
| ISP Dev info | DevId     | Isp Dev Id      |
|              | CreChnNum | 当前 Chn 数量       |
|              | DevMask   | Stitch Dev Mask |
|              | cmdq      | Cmdq 指针         |
|              | En        | ISR 中断使能标志      |
|              | Mode      | ISR 中断模式        |
|              | num       | ISR ID          |
|              | VsyncCnt  | ISR ID          |

| 参数 |              | 描述                 |
|----|--------------|--------------------|
|    | FrameDoneCnt | ISR Frame Done Cnt |
|    | DropCnt      | ISR Frame Drop Cnt |

| 参数           |                                                                  | 描述                          |
|--------------|------------------------------------------------------------------|-----------------------------|
| ISP Chn info | DevId                                                            | Isp Dev Id                  |
|              | ChnId                                                            | Isp Chn Id                  |
|              | start                                                            | Chn Start Flag              |
|              | SnrId                                                            | Bind sensorId               |
|              | Sync3A                                                           | Sync 3A type                |
|              | chnNum                                                           | Muti chn Num                |
|              | Crop                                                             | Input Crop Size             |
|              | InSize                                                           | Input Size                  |
|              | pixel                                                            | Input pixel format          |
|              | Stride                                                           | Input Buffer Stride         |
|              | Atom                                                             | 底层拿住 buffer 数量              |
|              | Atom0                                                            | 释放 buffer 后底层没有 buffer 数量   |
|              | Rot                                                              | Rotation 角度                 |
|              | bMirror/Flip                                                     | 水平/竖直 翻转                    |
|              | 3DNRLevel                                                        | 3DNR Level                  |
|              | HdrMode                                                          | HDR mode                    |
|              | PreCnt/EnqCnt/<br>BarCnt/<br>checkin/checkout/<br>DeqCnt/DropCnt | Callback 接口执行次数             |
|              | EnqOTNull                                                        | Enq 时 OutBuffer 为 Null 次数统计 |

| 参数                      |          | 描述                       |
|-------------------------|----------|--------------------------|
| ISP OutPut<br>Port info | DevId    | Dev Id                   |
|                         | ChnId    | Plane id                 |
|                         | PortID   | Port Id                  |
|                         | Bindtype | 与后级绑定模式                  |
|                         | Enable   | Port enable flag         |
|                         | Pixel    | Port output pixel format |
|                         | PortCrop | output Crop Size         |

| 参数 |           | 描述                     |
|----|-----------|------------------------|
|    | Stride    | Output Stride          |
|    | GetCnt    | 尝试获取 OutPut buffer 数量  |
|    | Failcnt   | 获取 OutPut buffer 失败数量  |
|    | FinishCnt | 处理完 OutPut buffer 数量   |
|    | fps       | Output port frame rate |

## 5.2. echo

# echo help > /proc/mi\_modules/mi\_isp/mi\_isp0

```
debugmutex [/];
stopchnl [chnid ON/OFF];
dumptaskfile [chnid Cnt /path/ passid bOnlyDumpResChange bdumpport];
setatom [chnid AtomValue];
clearbuf [chnid portid bClearInput bClearOutput bClearDoneBuff passid];
debuglv [level](1:isp api,2:flow,4:check irq done)
 [chnid][level] (1:check buffer,2:check loop,4:check frame pts,8:check fence done,16:check func time,32:check sidebandmsg)
enableport [chnid portid bEn];
skipframe [chnid skipnum];
protectport [chnid portid clientid bEn passid];
 clientid:isp:0x23, LDC:0x0e, scl(0,1,2):0x19, 0x1c, 0x1d
clearbindq [chnid bclear passid];
fpsth [chnid portid fpsint fpsfloat passid];
outputparam [pixel chnid portid pixelmode];
outputparam [compress chnid portid compressmode];
outputparam [crop chnid portid cropX cropY cropW cropH];
outputparam [all chnid portid pixelmode compressmode cropX cropY cropW cropH];
```

Echo help 查看可用命令

|      |                                                   |
|------|---------------------------------------------------|
| 功能   | 打印所有 channel 锁的使用 function 和 line 和 time          |
| 命令   | echo debugmutex > /proc/mi_modules/mi_isp/mi_isp0 |
| 参数说明 | 无                                                 |
| 举例   | 无                                                 |

|      |                                                                  |
|------|------------------------------------------------------------------|
| 功能   | Stop 指定 channel                                                  |
| 命令   | echo stopchnl [ChnID] [ON/OFF] > /proc/mi_modules/mi_isp/mi_isp0 |
| 参数说明 | [ChnId] :通道号<br>[ON/OFF]<br>ON:stop chn<br>OFF:start chn         |
| 举例   | echo stopchnl 0 ON > /proc/mi_modules/mi_isp/mi_isp0             |

|      |                                                                                                                           |
|------|---------------------------------------------------------------------------------------------------------------------------|
| 功能   | Dump 选定 channel 的 output 数据，并保存在/path 路径下                                                                                 |
| 命令   | echo dumptaskfile [chnid, Cnt, /path/, bOnlyDumpResChange, bdumpport] > /proc/mi_modules/mi_isp/mi_isp0                   |
| 参数说明 | Chnid: channel id                                                                                                         |
|      | Cnt: dump 数量                                                                                                              |
|      | Path: 存放 dump 出来文件的路径                                                                                                     |
|      | bOnlyDumpResChange: 有时候有些花图在切分辨率时产生，敲 dump 命令和切分辨率操作不好配合，可以先设置该参数，然后再切分辨率，当程序识别到分辨率变化时就会将分辨率变化后的那张 buffer dump 出来。（非必选参数） |
|      | Bdumpport: 只 dump 某一个 portid，默认是 dump 所有在输出的 port。（非必选参数）                                                                 |
| 举例   | echo dumptaskfile 0 1 /mnt > /proc/mi_modules/mi_isp/mi_isp0                                                              |

|      |                                                                                                                                             |
|------|---------------------------------------------------------------------------------------------------------------------------------------------|
| 功能   | 设置 Dev 的 atomvalue                                                                                                                          |
| 命令   | echo setatom [AtomValue] > /proc/mi_modules/mi_isp/mi_isp0                                                                                  |
| 参数说明 | AtomValue: input realtime mode 下 Driver 最大持有 buffer 数量                                                                                      |
| 举例   | echo setatom 3 > /proc/mi_modules/mi_isp/mi_isp0<br>ps: 有时候底层卡住，可以尝试增大一下这个 atom，可以再塞一张 buffer 给 driver，重新 trig 一次，看是否可以恢复，或者掉帧的时候，加大是否可以满帧。 |

|      |                                                                                                           |
|------|-----------------------------------------------------------------------------------------------------------|
| 功能   | 清除指定 channel port 的 input/output/done buffer                                                              |
| 命令   | echo clearbuf [chnid, portid bClearInput, bclearoutput, bcleardonebuff] > /proc/mi_modules/mi_isp/mi_isp0 |
| 参数说明 | Chnid: channel id                                                                                         |
|      | Portid: port id                                                                                           |
|      | bClearInput: 清除 input 的 buffer                                                                            |
|      | bclearoutput: 清除 output 的 buffer                                                                          |
|      | cleardonebuff: 清除 done 的 buffer                                                                           |
| 举例   | echo clearbuf 0 0 1 0 0 > /proc/mi_modules/mi_isp/mi_isp0                                                 |

|      |                                                                                                                                  |
|------|----------------------------------------------------------------------------------------------------------------------------------|
| 功能   | 设置 debug level                                                                                                                   |
| 命令   | echo debuglv [level] >/proc/mi_modules/mi_isp/mi_isp0                                                                            |
|      | echo debuglv [chnid][level] >/proc/mi_modules/mi_isp/mi_isp0                                                                     |
| 参数说明 | Chnid: channel id                                                                                                                |
|      | Level:<br>单个 level 参数时: 1:isp api,2:flow,4:check irq done<br>两个个参数时: 1:check buffer,2:check loop,4:check frame pts,8:check fence |



|    |                                                                                                           |
|----|-----------------------------------------------------------------------------------------------------------|
|    | done,16:check func time,32:check sidebandmsg,64:check zoominfo                                            |
| 举例 | echo debuglv 4 > /proc/mi_modules/mi_isp/mi_isp0<br>//打开 check irq done 相关的 MI 打印                         |
|    | echo debuglv 0 3 > /proc/mi_modules/mi_isp/mi_isp0<br>/打开 channel0 level3 的打印, 打印 buffer info 和 loop info |

|      |                                                                   |
|------|-------------------------------------------------------------------|
| 功能   | 主动 drop frame 数量                                                  |
| 命令   | echo skipframe [chnid, skipnum] > /proc/mi_modules/mi_isp/mi_isp0 |
| 参数说明 | Chnid: channel id                                                 |
|      | skipnum: drop frame count                                         |
| 举例   | echo skipframe 0 60 > /proc/mi_modules/mi_isp/mi_isp0             |

|      |                                                                                                  |
|------|--------------------------------------------------------------------------------------------------|
| 功能   | 保护 outputport buffer                                                                             |
| 命令   | echo protectport [chnid portid clientid ben] > /proc/mi_modules/mi_isp/mi_isp0                   |
| 参数说明 | Chnid: channel id                                                                                |
|      | Portid: port id                                                                                  |
|      | Clientid: isp:0x23, LDC:0x0e, scl(0,1,2):0x19, 0x1c, 0x1d                                        |
|      | ben: 使能                                                                                          |
| 举例   | echo protectport 0 0 0x23 1 > /proc/mi_modules/mi_isp/mi_isp0<br>//保护 channel0 pass0 port0 的 isp |

|      |                                                                                                                                                                                                                                                     |
|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 功能   | 将 Bind Q 中缓存的 buffer 消耗掉                                                                                                                                                                                                                            |
| 命令   | echo clearbindq [chnid bclear] > /proc/mi_modules/mi_isp/mi_isp0                                                                                                                                                                                    |
| 参数说明 | Chnid: channel id                                                                                                                                                                                                                                   |
|      | bclear: 是否 clear                                                                                                                                                                                                                                    |
| 举例   | echo clearbindq 0 1 > /proc/mi_modules/mi_isp/mi_isp0<br>//clear channel0 bindQ<br>将 Bind Q 中缓存的 buffer 消耗掉。<br>有时候前端拿不到 buffer, 发现是 ISP BindQ 塞满, 判断 ISP 是否拿的不够快, 将 bindQ 清空, 看 BindQ 中是否还会累加上来, 如果还累加则说明 ISP 拿的不够快, 如果不再增加, 说明是之前一次异常塞在里面一直没有消耗掉。 |

|      |                                                                             |
|------|-----------------------------------------------------------------------------|
| 功能   | 设置打印帧率, 低于该值打印出来                                                            |
| 命令   | echo fpsth [chnid portid fpsint fpsfloat] > /proc/mi_modules/mi_isp/mi_isp0 |
| 参数说明 | Chnid: channel id                                                           |
|      | Portid: port id                                                             |
|      | fpsint: 帧率整数位                                                               |

|    |                                                                                                  |
|----|--------------------------------------------------------------------------------------------------|
|    | fpsfloat: 帧率小数位                                                                                  |
| 举例 | echo fpsth 0 0 30 30 > /proc/mi_modules/mi_isp/mi_isp0<br>// channel0 port0 中小于 30.30 的 fps 打印出来 |

|      |                                                                        |
|------|------------------------------------------------------------------------|
| 功能   | 设置 port 是否 enable                                                      |
| 命令   | echo enableport [chnid, portid, bEn] > /proc/mi_modules/mi_isp/mi_isp0 |
| 参数说明 | Chnid: channel id                                                      |
|      | Portid: port id                                                        |
|      | bEn: 1/0                                                               |
| 举例   | echo enableport 0 0 1 > /proc/mi_modules/mi_isp/mi_isp0                |

|      |                                                                                                                       |
|------|-----------------------------------------------------------------------------------------------------------------------|
| 功能   | 设置 output port 参数                                                                                                     |
| 命令   | echo outputparam pixel [chnid, portid pixelmode] > /proc/mi_modules/mi_isp/mi_isp0                                    |
|      | echo outputparam compress [chnid, portid compressmode] > /proc/mi_modules/mi_isp/mi_isp0                              |
|      | echo outputparam crop [chnid, portid cropX cropY cropW cropH] > /proc/mi_modules/mi_isp/mi_isp0                       |
|      | echo outputparam all [chnid, portid pixelmode compressmode cropX cropY cropW cropH] > /proc/mi_modules/mi_isp/mi_isp0 |
| 参数说明 | Chnid: channel id                                                                                                     |
|      | Portid: port id                                                                                                       |
|      | Pixelmode: 输出的 pixel 类型, 填入 MI_SYS_PixelFormat_e 中对应的数值                                                               |
|      | Compressmode: 输出的压缩模式, 填入 MI_SYS_CompressMode_e 中对应的数值                                                                |
|      | cropX: 输出的端口裁剪 X 坐标                                                                                                   |
|      | cropY: 输出的端口裁剪 Y 坐标                                                                                                   |
|      | cropW: 输出的端口裁剪宽度                                                                                                      |
|      | cropH: 输出的端口裁剪高度                                                                                                      |
| 举例   | echo outputparam pixel 0 1 11 > /proc/mi_modules/mi_isp/mi_isp0                                                       |
|      | echo outputparam compress 0 1 5 > /proc/mi_modules/mi_isp/mi_isp0                                                     |
|      | echo outputparam crop 0 1 0 0 1920 1080 > /proc/mi_modules/mi_isp/mi_isp0                                             |
|      | echo outputparam all 0 1 11 5 0 0 1920 1080 > /proc/mi_modules/mi_isp/mi_isp0                                         |